



OBJECT ORIENTED STRUCTURAL MODELING

STATIC UML DESIGN

CHAPTER SEVEN



OUTLINE

- Class diagram
- Object diagram
- Collaboration diagram
- Architectural design

STRUCTURE DIAGRAMS

- Class Diagram
 - Describe the **structure** of the system in terms of **classes** and **objects**
 - Primary purpose during analysis workflow
 - to create a vocabulary that is used by both the analyst and users

DISCOVERING CLASSES

■ CRC Card

- A simple object-oriented analysis technique that **includes the users and developers** in the analysis process
- A CRC card is an index card with three parts:
 - ***Class*** -- name goes at the top of the card
 - ***Responsibilities*** -- as a list on the left side of the card
 - ***Collaborators*** -- as a list on the right side of the card

Class Name	
Responsibilities	Collaborators

Example:

Class name: Real estate owner	Superclass: Person	Subclasses: Apartment owner
Responsibilities	Collaborations	
Invites	Invitation, Person, List, ...	
Buys	Money, Shop, Food, Furniture, ...	
Cleans	Duster, Sponge, ...	

■ Class

- Represents a type of object being modeled
- One card per class

■ Responsibility

- Something that the class knows (keeps track of) or does
- These should be the high-level responsibilities.
- Not trying to list out all member functions here

■ Collaborator

- Another class that the current class has to work with to complete its responsibilities
- Could be a class that has information we need
- Could be a class that helps perform a task
- Typically, we list a class as a collaborator if we (the current class) need to **call** upon it to **help complete our own responsibilities**

DISCOVERING CLASSES (ALTERNATIVE)

- **Underline the nouns** in a problem statement.
 - Using the problem context and general knowledge about the problem domain **decide on the important nouns**.
 - Design and implement **classes** to represent the nouns.
- **Underline the verbs**.
 - Verbs related to a class may represent the **behavior** of the class.

CLASS NAMING RULE

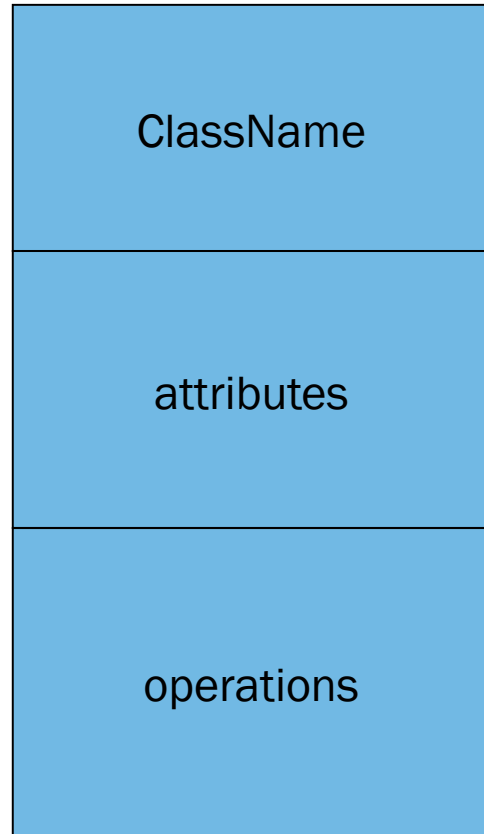
- A class name must always be either:
 - A **noun**, possibly qualified.
 - (Only for a deferred class describing a structural property) an **adjective**.
- Names of classes are written with **an initial capital letter** (like this: **Button**, not like this: **button**) and **have no spaces**.
- To show where a space would be, the next letter is capitalized (like this: **BarcodeReader**, not like this: **Barcode Reader**).

-
- Class names are **singular**, not plural.

.....CONT

- It is incorrect to name a class “Books”.
 - Although there may be many book objects, there is only one book class.
- Names of attributes and operations start with a **lower-case letter**, but **may have capital letters to indicate where the spaces** would be, if the name would normally have spaces.
- Names of **operations** are **followed by brackets ()** to distinguish them from names of attributes

CLASS



A *class* is a description of a set of **objects** that share the same **attributes**, **operations**, **relationships**, and **semantics**.

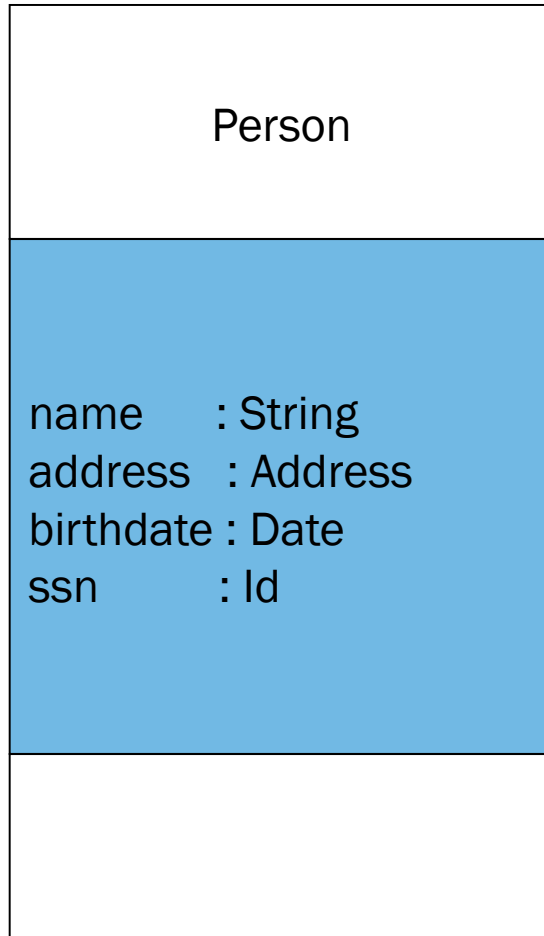
Graphically, a class is rendered as a rectangle, usually including its **name**, **attributes**, and **operations** in separate, designated compartments.

CLASS NAMES

ClassName
attributes
operations

- The *name of the class* is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.
- Underline if the class is static class

CLASS ATTRIBUTES



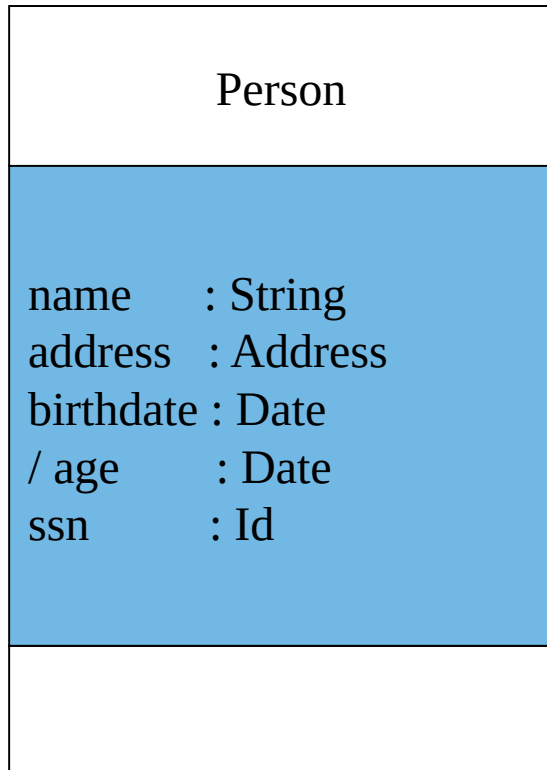
An *attribute* is a named property of a class that describes the object being modeled.

In the class diagram, attributes appear in the second compartment just below the name-compartment.

CLASS ATTRIBUTES ...

Attributes are usually listed in the form:

Visibility attributeName : Type



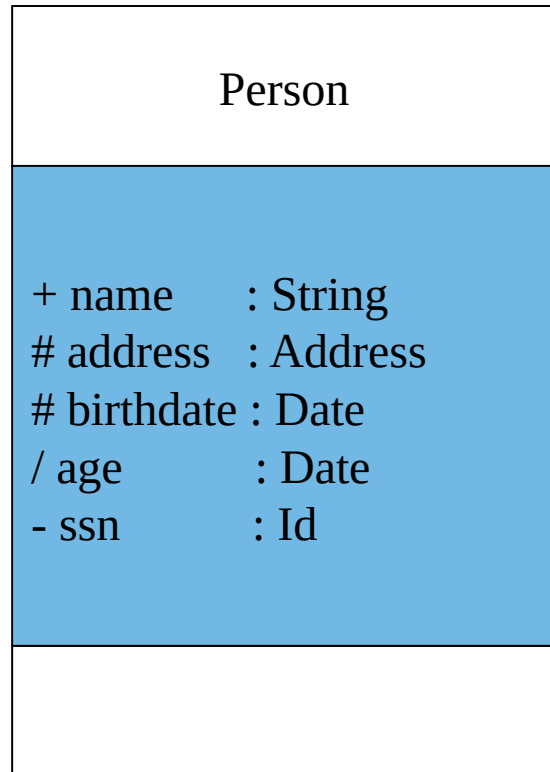
- A *derived* attribute is one that can be computed from other attributes, but doesn't actually exist.

For example,

a Person's age can be computed from his birth date. A derived attribute is labelled by a preceding '/' as in:

/ age : Date

CLASS ATTRIBUTES ...



Attributes can be:

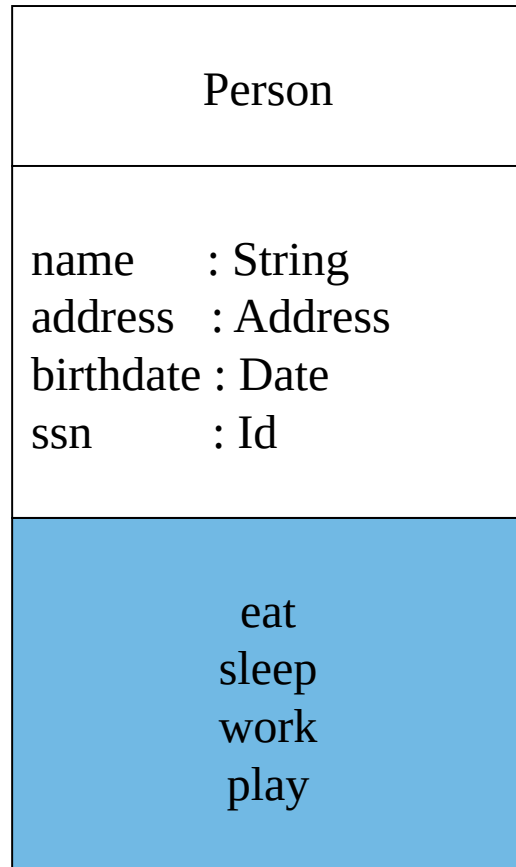
+ public

protected

- private

/ derived

CLASS OPERATIONS



Operations describe the class behavior and appear in the third compartment.

CLASS OPERATIONS ...

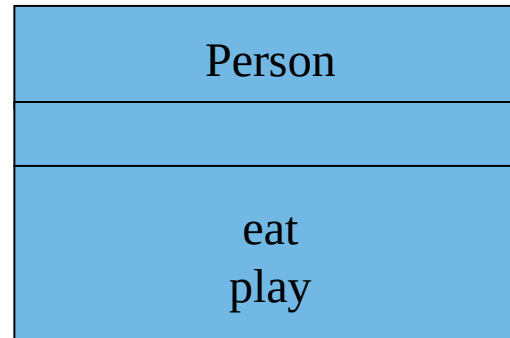
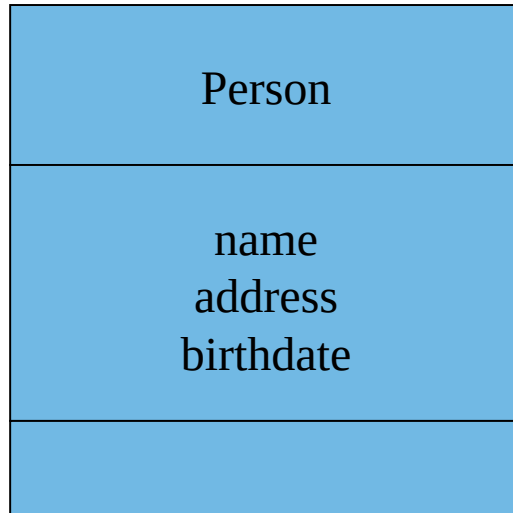
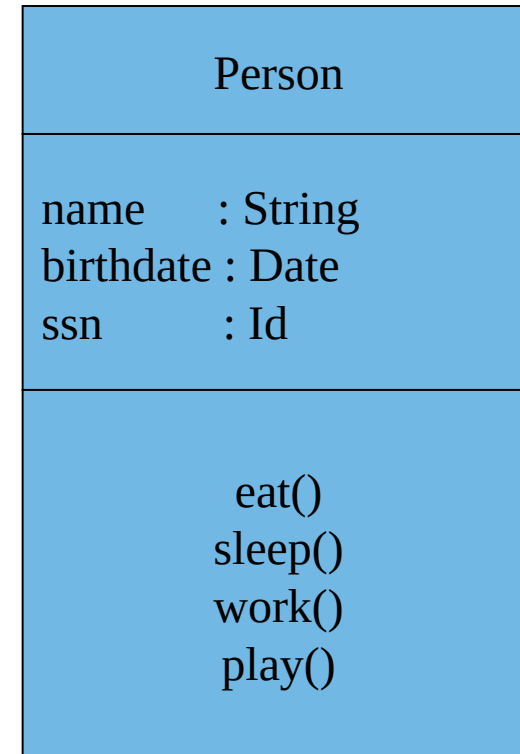
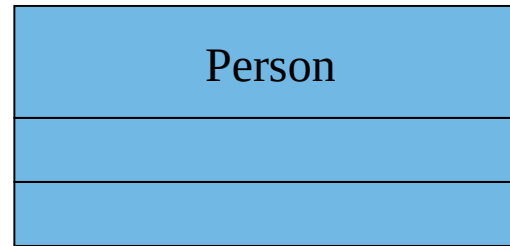
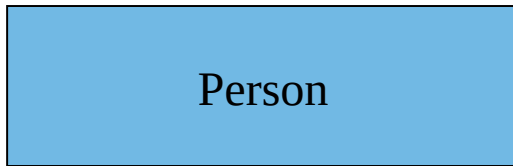
PhoneBook
<code>newEntry (n : Name, a : Address, p : PhoneNumber, d : Description)</code> <code>getPhone (n : Name, a : Address) : PhoneNumber</code>

Visibility **nameMethod(parameterName:dataType) : dataTypeforMethod**

You can specify an operation by stating its signature: listing the name, type, and default value of all parameters, and, in the case of functions, a return type.

DEPICTING CLASSES

When drawing a class, you needn't show attributes and operation in every diagram.



RELATIONSHIPS

- In UML, object interconnections (logical or physical), are modeled as relationships.
- There are Four kinds of relationships in UML Class diagram:
 - Associations
 - Dependencies
 - Generalizations
 - Realization

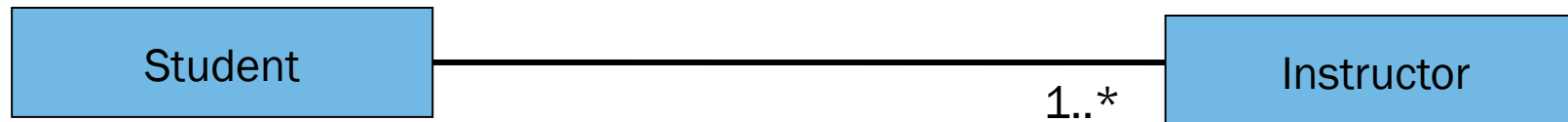
ASSOCIATION

- If two classes in a model need to communicate with each other, there must be link between them.
- An *association* denotes that link.



ASSOCIATION ...

- We can indicate the *multiplicity* of an association by adding *multiplicity adornments* to the line denoting the association.
- The example indicates that a *Student* has one or more *Instructors*:



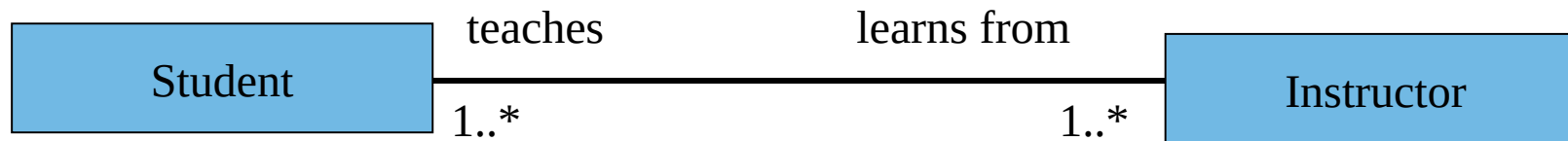
ASSOCIATION ...

The example indicates that every *Instructor* has one or more *Students*:



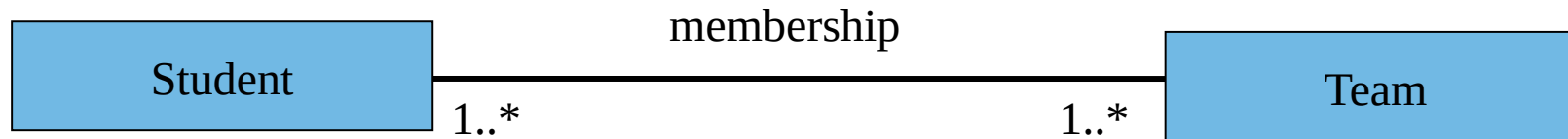
ASSOCIATION ...

We can also indicate the behavior of an object in an association (*i.e.*, the *role* of an object) using *rolenames*.



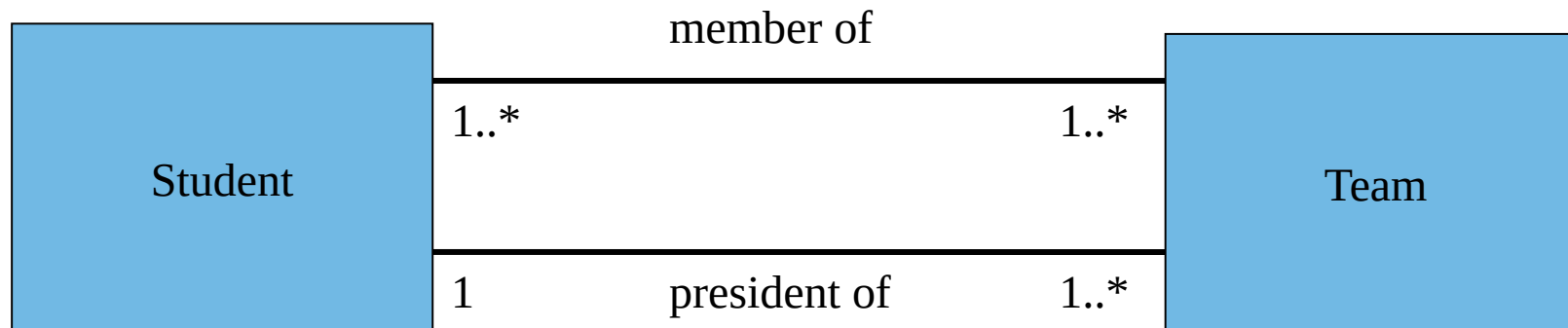
ASSOCIATION ...

We can also name the association.



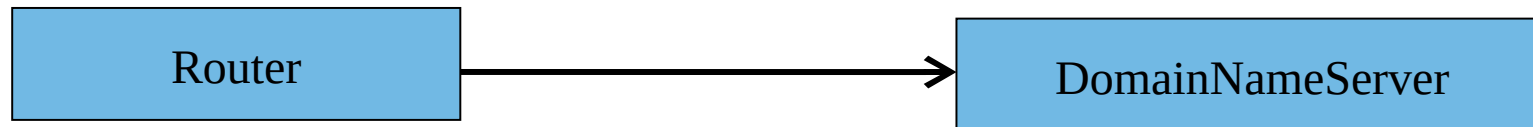
ASSOCIATION ...

We can specify dual associations.



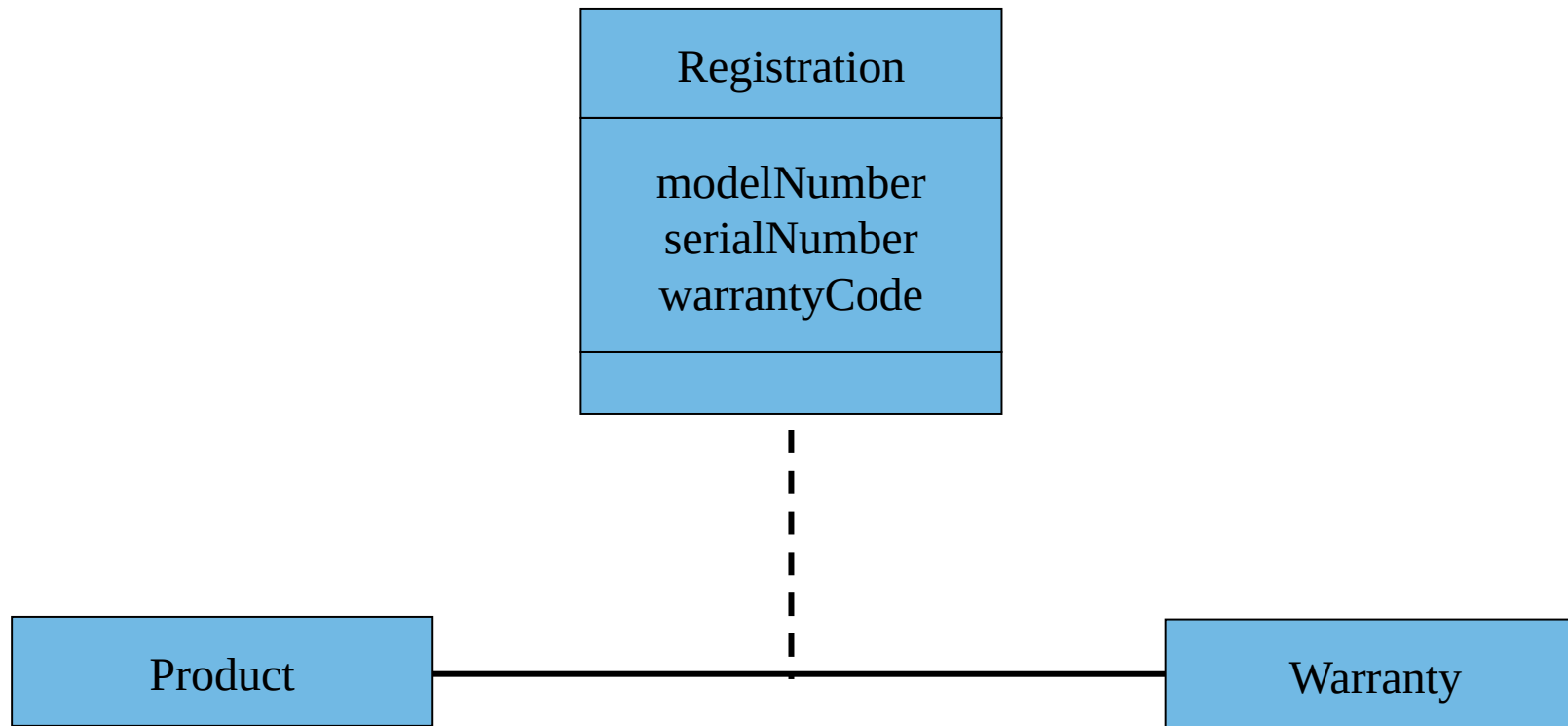
ASSOCIATION ...

- We can constrain the association relationship by defining the *navigability* of the association.
- Here, a *Router* object requests services from a *DNS* object by sending messages to (invoking the operations of) the server.
- The direction of the association indicates that the server has no knowledge of the *Router*.



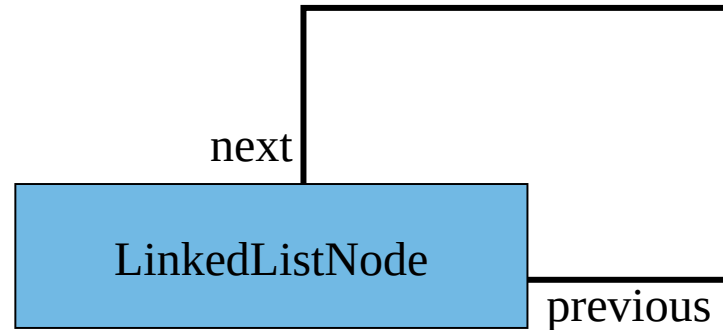
ASSOCIATION ...

- Associations can also be objects themselves, called *link classes* or an *association classes*.



ASSOCIATION ...

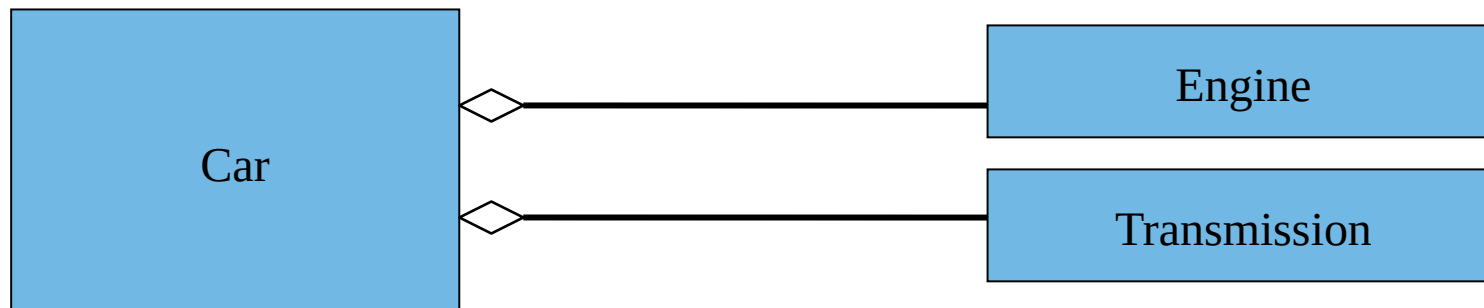
- A class can have a *self association*.
- This means that some instances of the class may be related to some other instances of the class.



ASSOCIATION ...

Aggregation

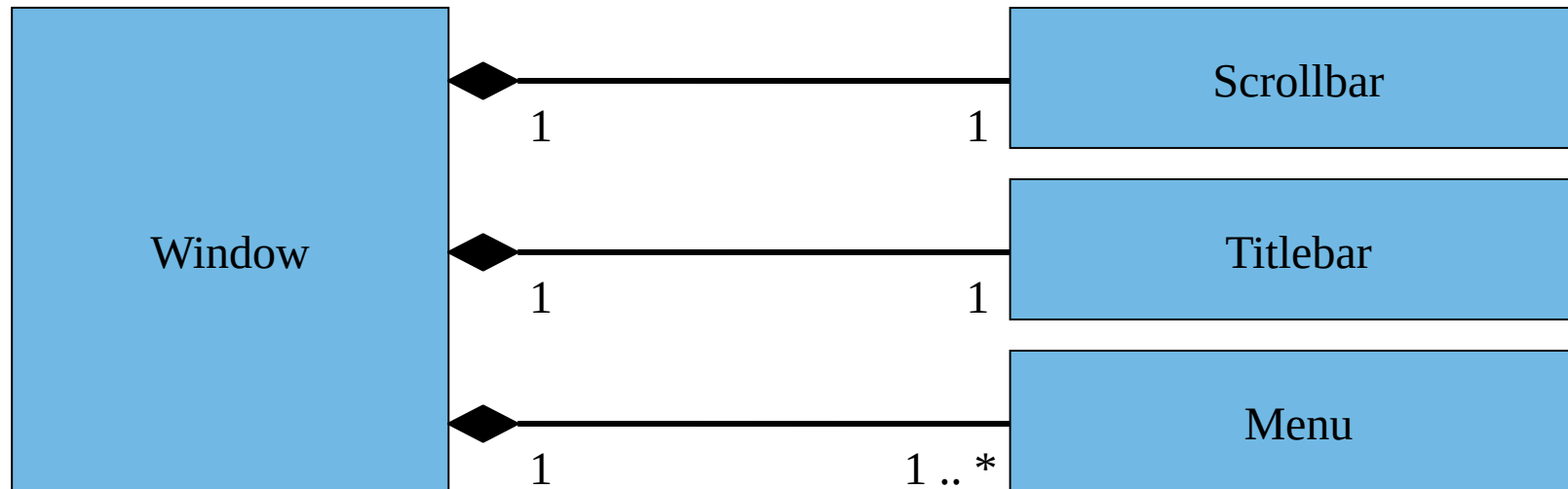
- specifies a whole-part relationship between an aggregate (a whole) and a constituent part, where the part can exist independently from the aggregate.
- *The contained classes are not strongly dependent on the lifecycle of the container.* Aggregations are denoted by a hollow-diamond adornment on the association.



ASSOCIATION ...

Composition

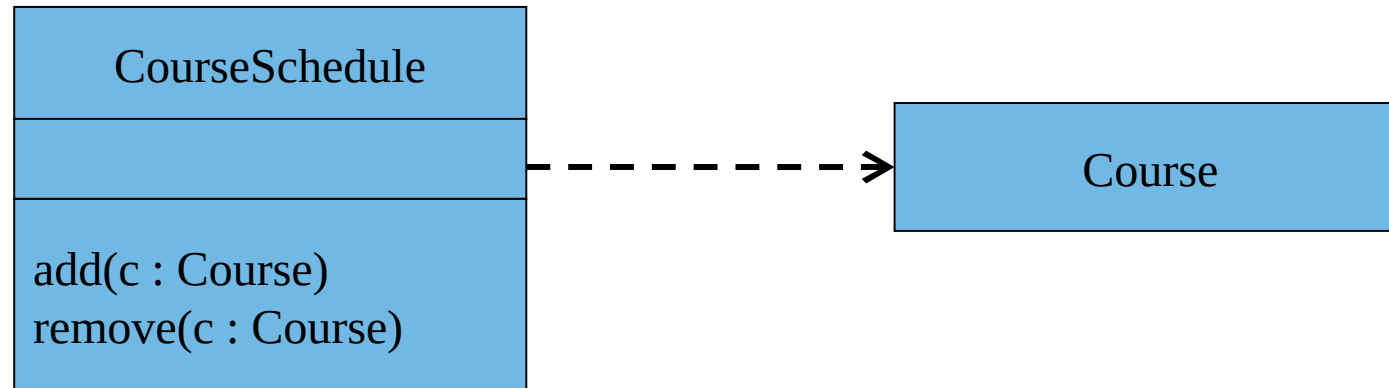
- indicates a strong ownership and coincident lifetime of parts by the whole (*i.e.*, they live and die as a whole). Compositions are denoted by a filled-diamond adornment on the association.



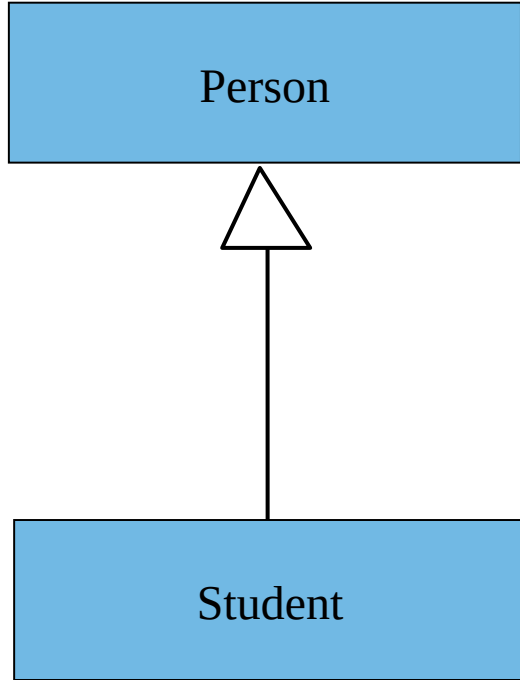
DEPENDENCY

Dependency

- indicates a semantic relationship between two or more elements.
- The dependency from *CourseSchedule* to *Course* exists because *Course* is used in both the **add** and **remove** operations of *CourseSchedule*.



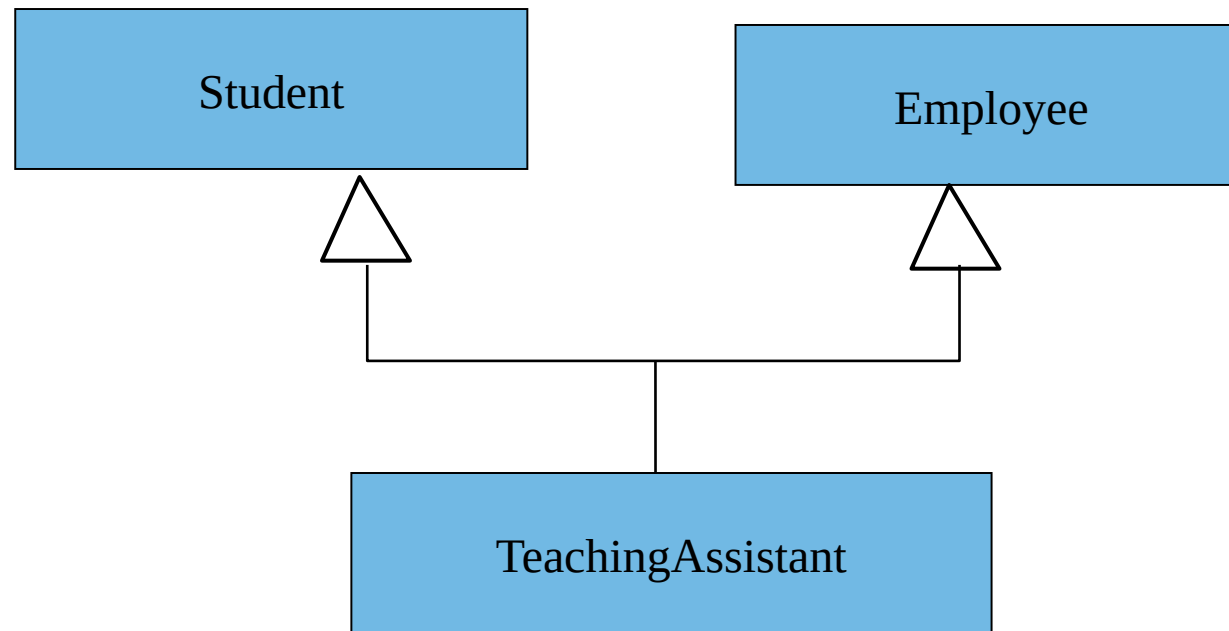
GENERALIZATION



- A **generalization** connects a **subclass** to its **superclass**.
- It denotes an **inheritance** of attributes and behavior from the superclass to the subclass and indicates a specialization in the subclass of the more general superclass.

GENERALIZATION

- UML permits a class to inherit from multiple super classes, although some programming languages (*e.g.*, Java) do not permit multiple inheritance.



TYPES OF CLASSES

- Entity classes
- Interface or Boundary classes
- Abstract classes
- Control classes

ENTITY CLASSES

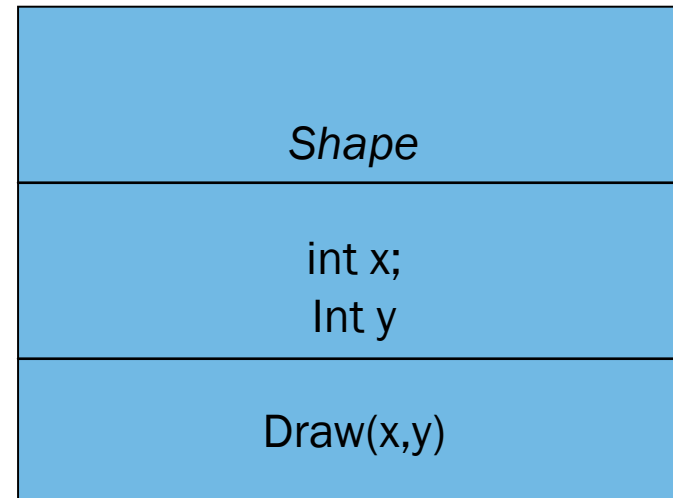
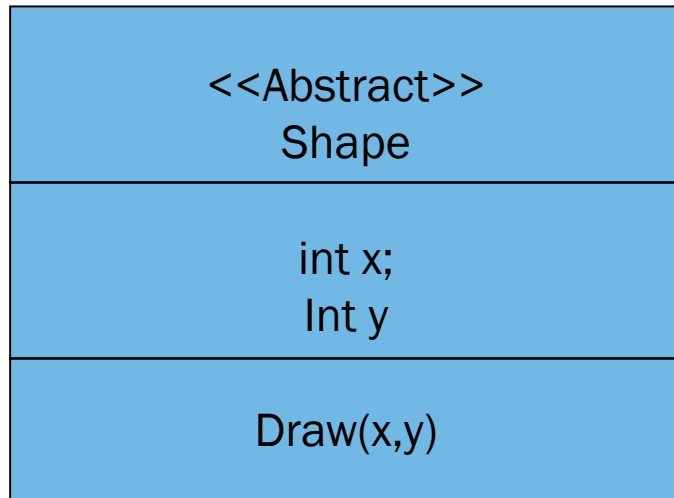
- Represent real-world items
 - Such as people, thing and so on
- The entities represented on an entity-relationship diagram

INTERFACE OR BOUNDARY CLASSES

- Provide a means for users to work with the system
- Human interfaces may be a display, window, Web form, dialogue box, touch-tone telephone, or other **way** for **users** to **interact** with the **system**
- System interfaces involve sending data to or receiving data from others
 - The attributes of these classes are those **found on the display or report.**
 - The methods are those required to **work with the display, or produce the report.**

ABSTRACT CLASSES

- Linked to concrete classes in a generalization/ specialization relationship
- Cannot be directly instantiated



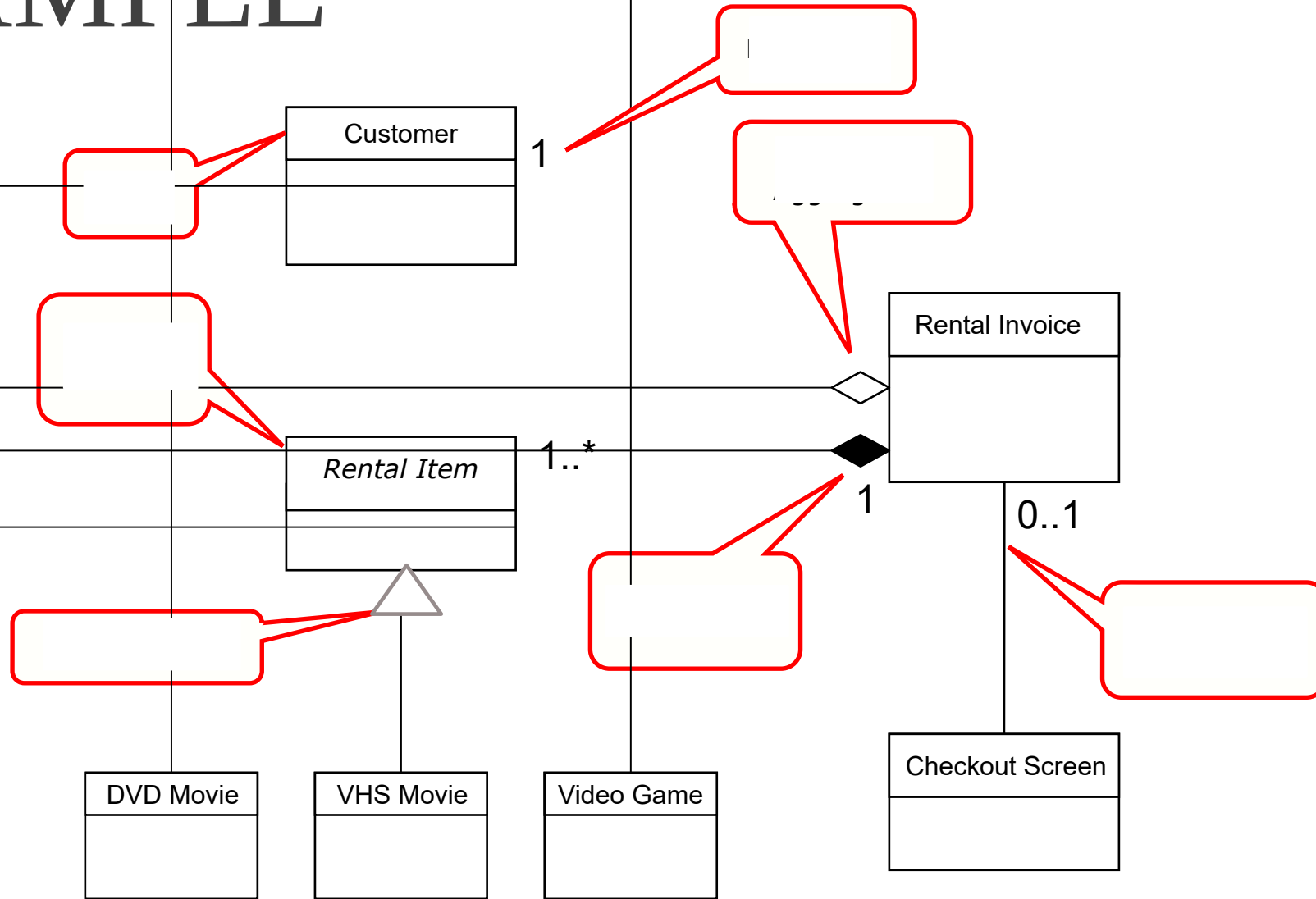
CONTROL CLASSES

- Used to control the **flow of activities**
- Many small control classes can be used to achieve classes that are reusable
- The rules for creating sequence diagrams are that all **interface** classes must be connected to a **control** class and in a similar fashion all **entity** classes must be connected to a **control** class.

FOUR RULES APPLY TO THEIR COMMUNICATION

- **Actors** can only talk to **boundary** objects.
- **Boundary** objects can only talk to **controllers** and **actors**.
- **Entity** objects can only talk to **controllers**.
- **Controllers** can talk to **boundary** objects and **entity** objects, and to other controllers, but **not** to **actors**

EXAMPLE

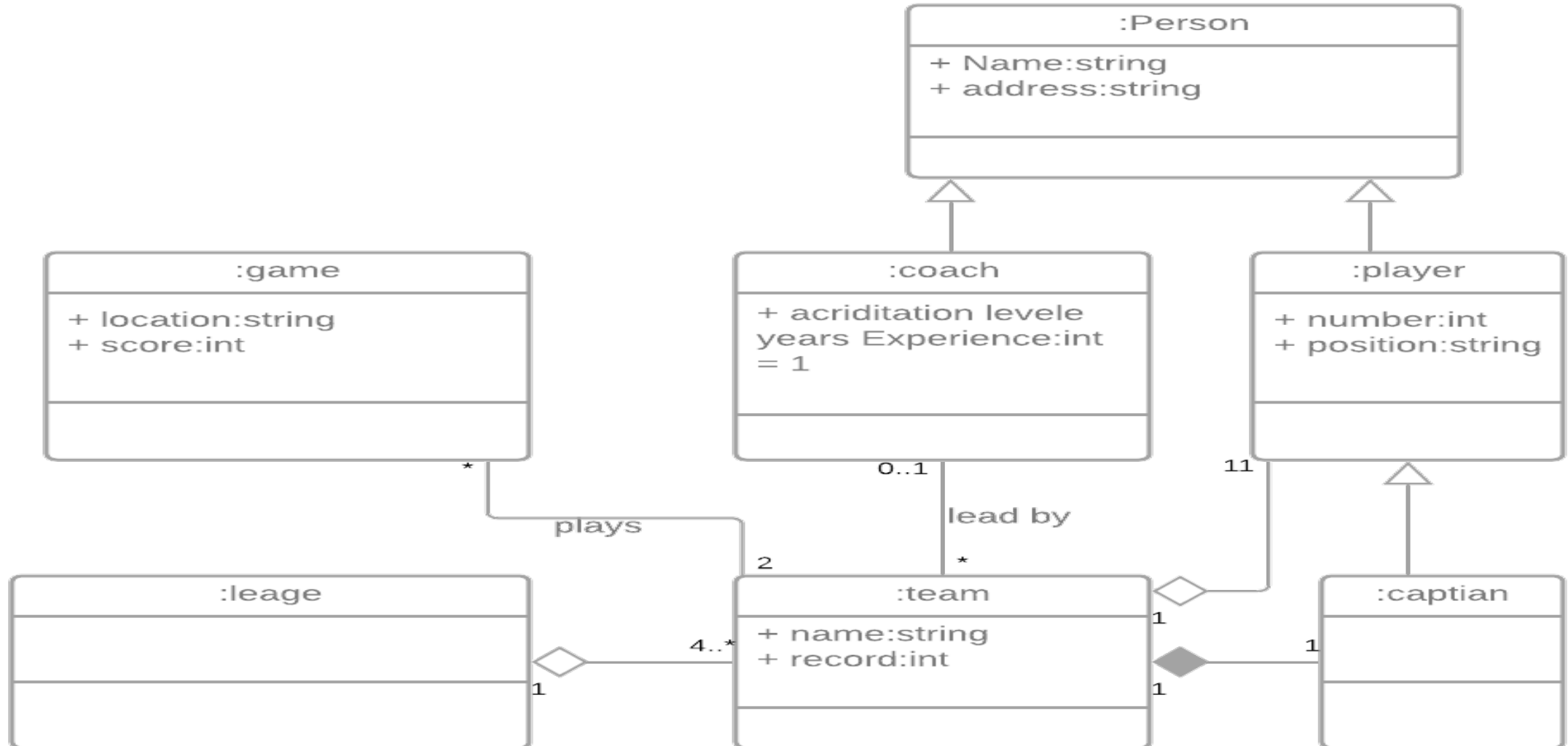


EXERCISE

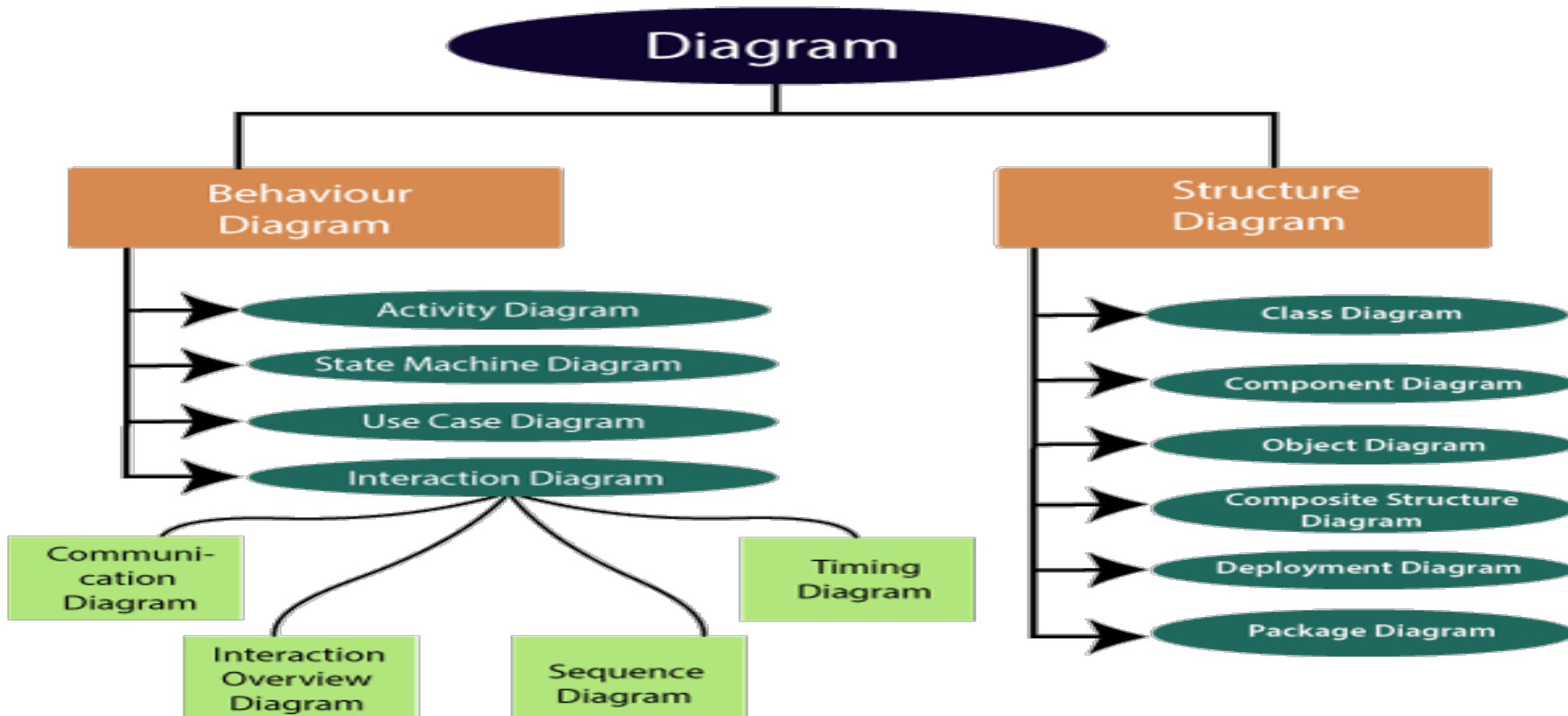
- Draw a UML Class Diagram representing the following elements from the problem domain for a soccer league.

A soccer league is made up of at least four soccer teams. Each soccer team is composed of eleven players, and one player captains the team. A team has a name and a record. Players have a number and a position. Soccer teams play games against each other. Each game has a score and a location. Teams are sometimes lead by a coach. A coach has a level of accreditation and a number of years of experience, and can coach multiple teams. Coaches and players are people, and people have names and addresses.

POSSIBLE ANSWER



UML DIAGRAM TYPES





UML OBJECT DIAGRAM

INTRODUCTION

- Object diagrams represent an **instance of a class diagram**.
- The **basic concepts** are **similar** for class diagrams and object diagrams.
- **Object diagrams also represent the static view of a system but this static view is a snapshot of the system at a particular moment.**
- Object diagrams are used to render a set of **objects and their relationships** as an instance.

PURPOSE OF OBJECT DIAGRAM

- But an object diagram represents **an instance at a particular moment** which is concrete in nature.
- It means the ***object diagram is more close to the actual system behavior.***
- The purpose is to capture the static view of a system at a particular moment.
- So the purpose of the object diagram can be summarized as:
 - Object relationships of a system
 - Static view of an interaction.
 - Understand object behavior and their relationship from practical perspective

BASIC OBJECT DIAGRAM SYMBOLS AND NOTATIONS

Object Names:

- Every object is actually symbolized like a **rectangle**, that offers the name from the object and its class **underlined** as well as divided with a **colon**.

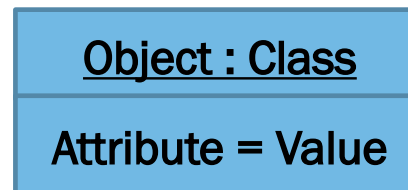


Object: Class

BASIC OBJECT DIAGRAM SYMBOLS AND NOTATIONS ...

Object Attributes:

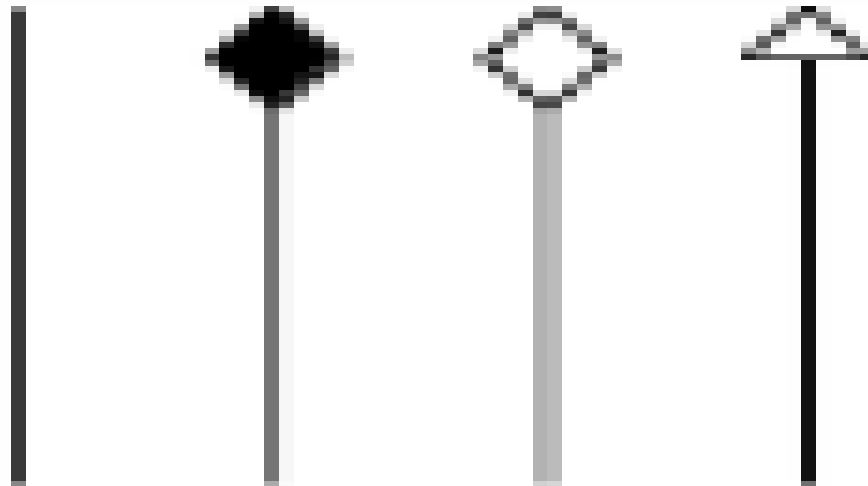
- Similar to classes, you are able to list **object attributes** inside a separate compartment. However, unlike classes, **object attributes should have values assigned for them.**



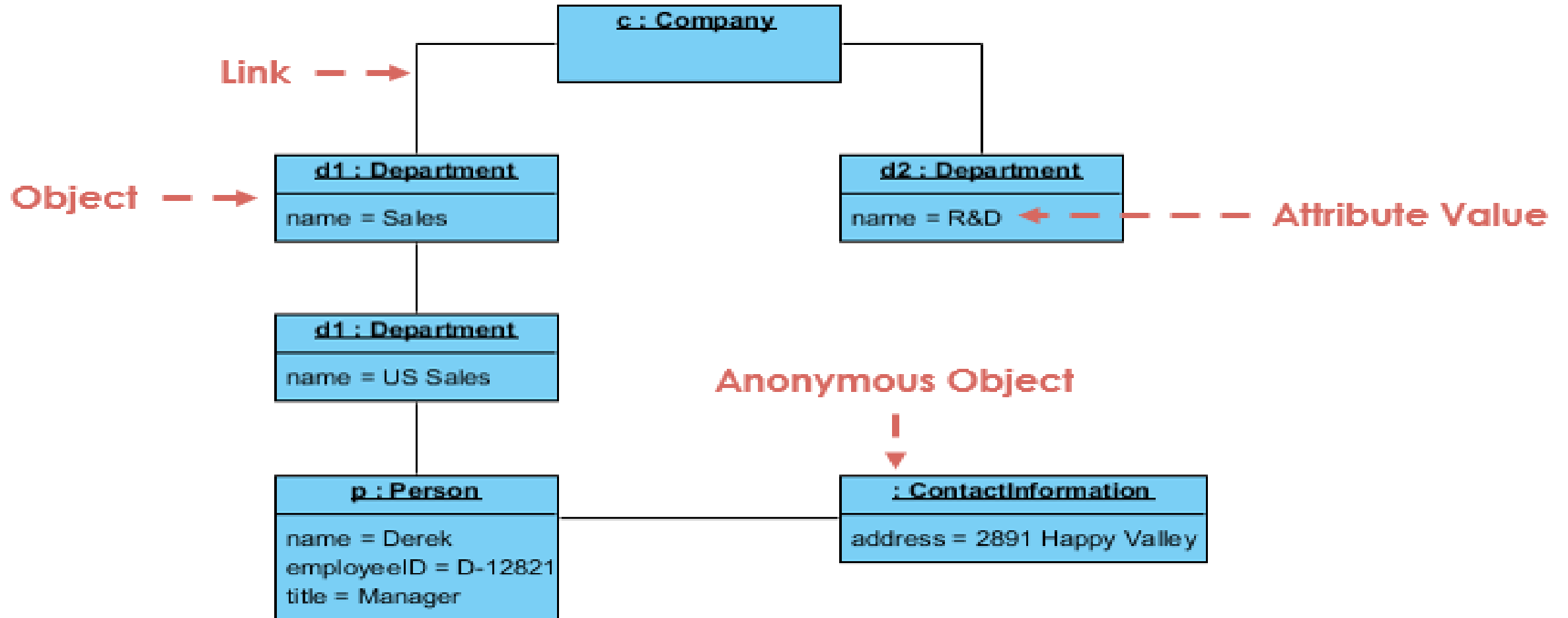
BASIC OBJECT DIAGRAM SYMBOLS AND NOTATIONS

Links:

- Links tend to be instances associated with associations. You can draw a link while using the lines utilized in class diagrams.



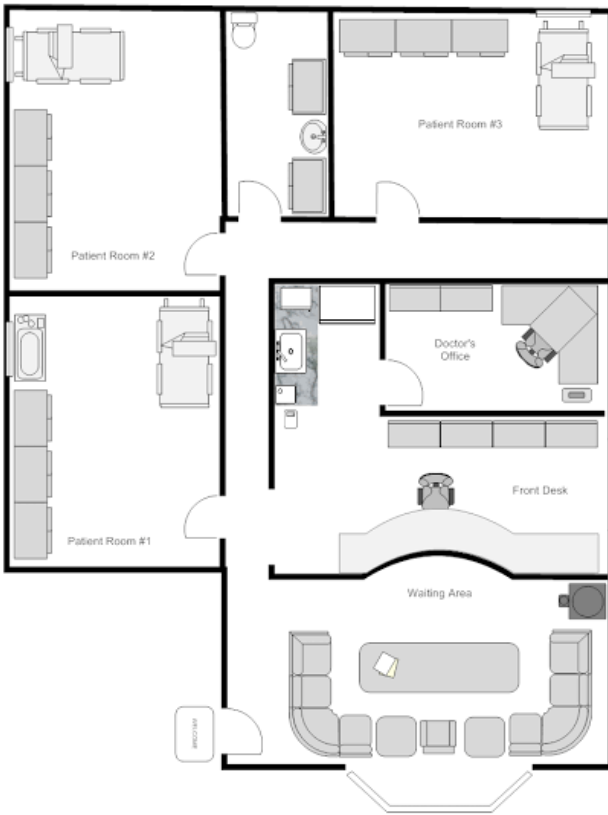
EXAMPLE 1



CLASS VS OBJECT DIAGRAM

- Both diagrams are made **of same basic elements but in different form.**
- In class diagram elements are in **abstract form to represent the blue print** and in object diagram the elements are in **concrete form to represent the real world object.**
- To capture a particular system, **numbers of class diagrams are limited.** But if we consider object diagrams then **we can have unlimited number of instances which are unique in nature.**

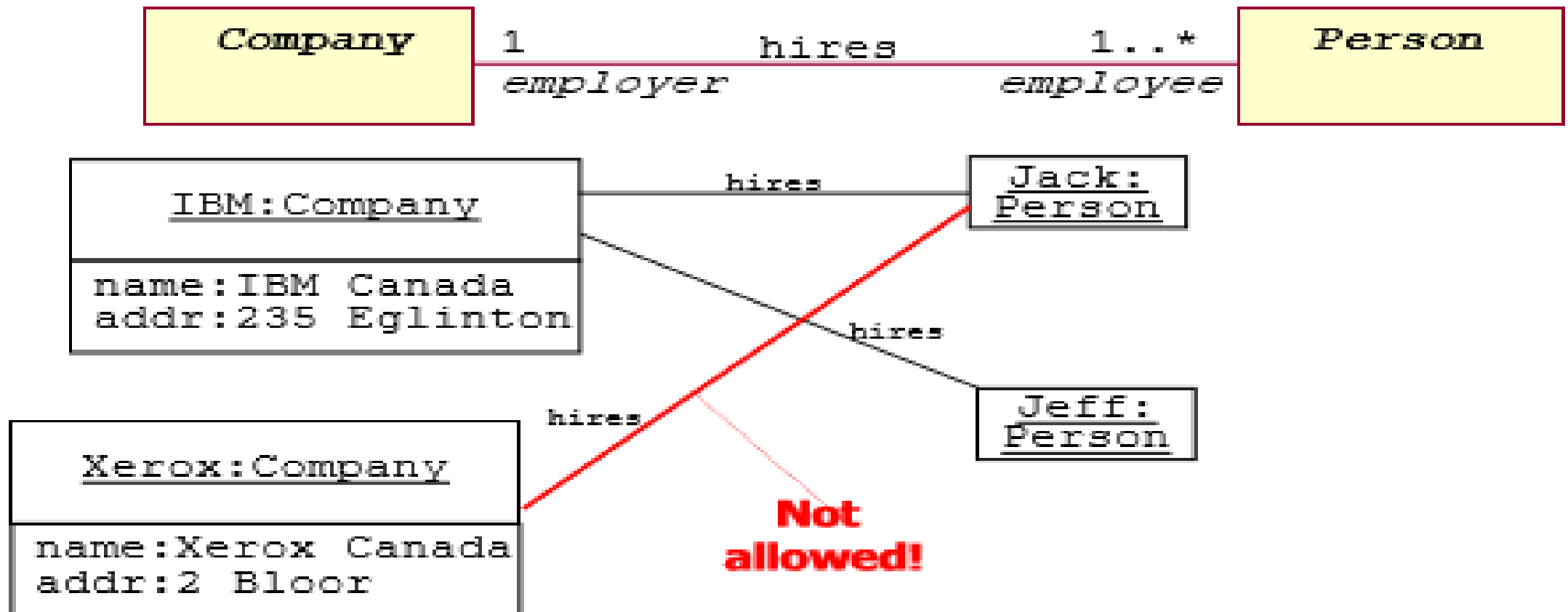
CLASS VS OBJECT DIAGRAM ...



CLASS VS OBJECT DIAGRAM ...

- Class become live when objects are created from them
- Object diagrams use notation that is similar to that used in class diagrams.
- Class diagrams show the actual classifiers and their relationships in a system
- Object diagrams show specific instances of those classifiers and the links between those instances at a point in time.
- You can create object diagrams by instantiating the classifiers in class, deployment, component, and use-case diagrams.

EXAMPLE 2



WHAT IS SOFTWARE ARCHITECTURE?

- *Software architecture* is the process of designing the **global organization** of a software system, including dividing software into **subsystems**, deciding how these will **interact**, and determining their **interfaces**.
- The *architect* is the person in charge of the whole project.
 - He or she has the responsibility to make sure that the building will be solid, cost-effective and satisfactory to the client.

THE IMPORTANCE OF DEVELOPING AN ARCHITECTURAL MODEL

- There are four **main reasons** why you need to develop an architectural model:
 - **To enable everyone to better understand the system.**
 - As a system becomes more and more complex, making it **understandable** is an increasing challenge.
 - This is especially true for **large, distributed systems** that use sophisticated technology.
 - **To allow people to work on individual pieces of the system.**
 - **To prepare for extension of the system.**
 - **To facilitate reuse and reusability.**
 - The architectural model makes each system component visible.
 - This is an important benefit since it encourages **reuse**.
 - By **analyzing the architecture**, you can **discover** those **components** that can be obtained from past projects or from third parties.

CONTENTS OF A GOOD ARCHITECTURAL MODEL

- ❑ A system's architecture will often be expressed in terms of several different *views*.
- ❑ These can include:
 - ▶ The logical breakdown into subsystems.
 - ▶ This is often shown using **package diagrams**
 - ▶ The interfaces among the subsystems must also be carefully described.
 - ▶ The dynamics of the interaction among components at run time,
 - ▶ Expressed using **interaction or activity diagrams**.
 - ▶ The data that will be shared among the subsystems
 - ▶ Expressed using **class diagrams**.
 - ▶ The components that will exist at run time, and the machines or devices on which they will be located.
 - ▶ This information can be expressed using **component diagram**.

DESCRIBING AN ARCHITECTURE USING UML

- All **UML** diagrams can be useful to **describe aspects** of the **architectural model**.
- Remember that the goal of architecture is to **describe** the **system** at a **very high level**, with emphasis on software **components** and their **interfaces**.
- Three other types of **UML diagram** are particularly important for **architecture modeling**:
 - Component diagrams, Package diagrams and deployment diagrams.



COMPONENT DIAGRAMS

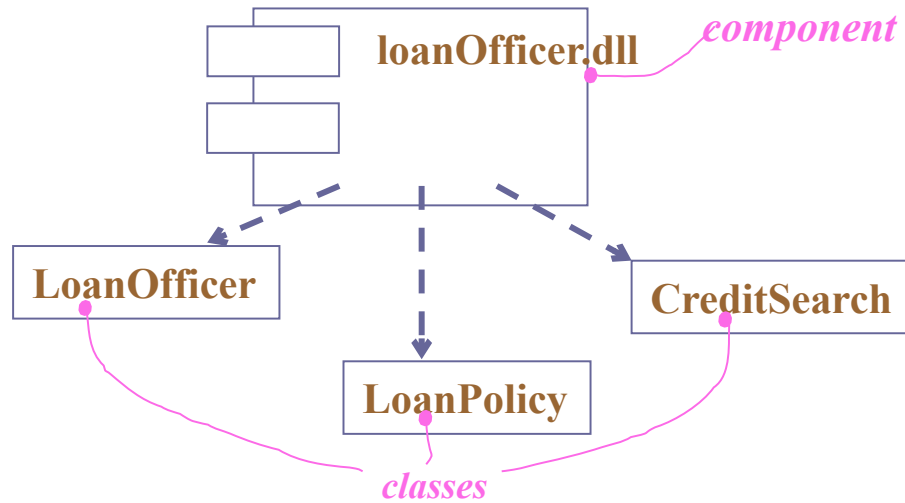


COMPONENT DIAGRAM

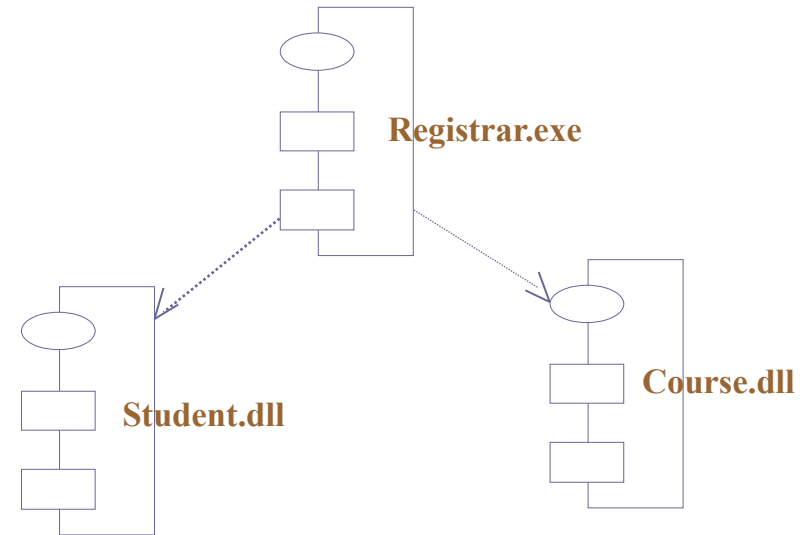
- ❑ Shows a set of components and their relationships.
- ❑ Represents the static implementation view of a system.
- ❑ Node corresponds to component.
- ❑ Edge represent relationship (the relationship can be read as “uses services of”)
- ❑ A component is a set of classes with well-defined interfaces
- ❑ Components map to one or more classes, interfaces, or collaborations.

COMPONENT DIAGRAM

Mapping of Components into Classes



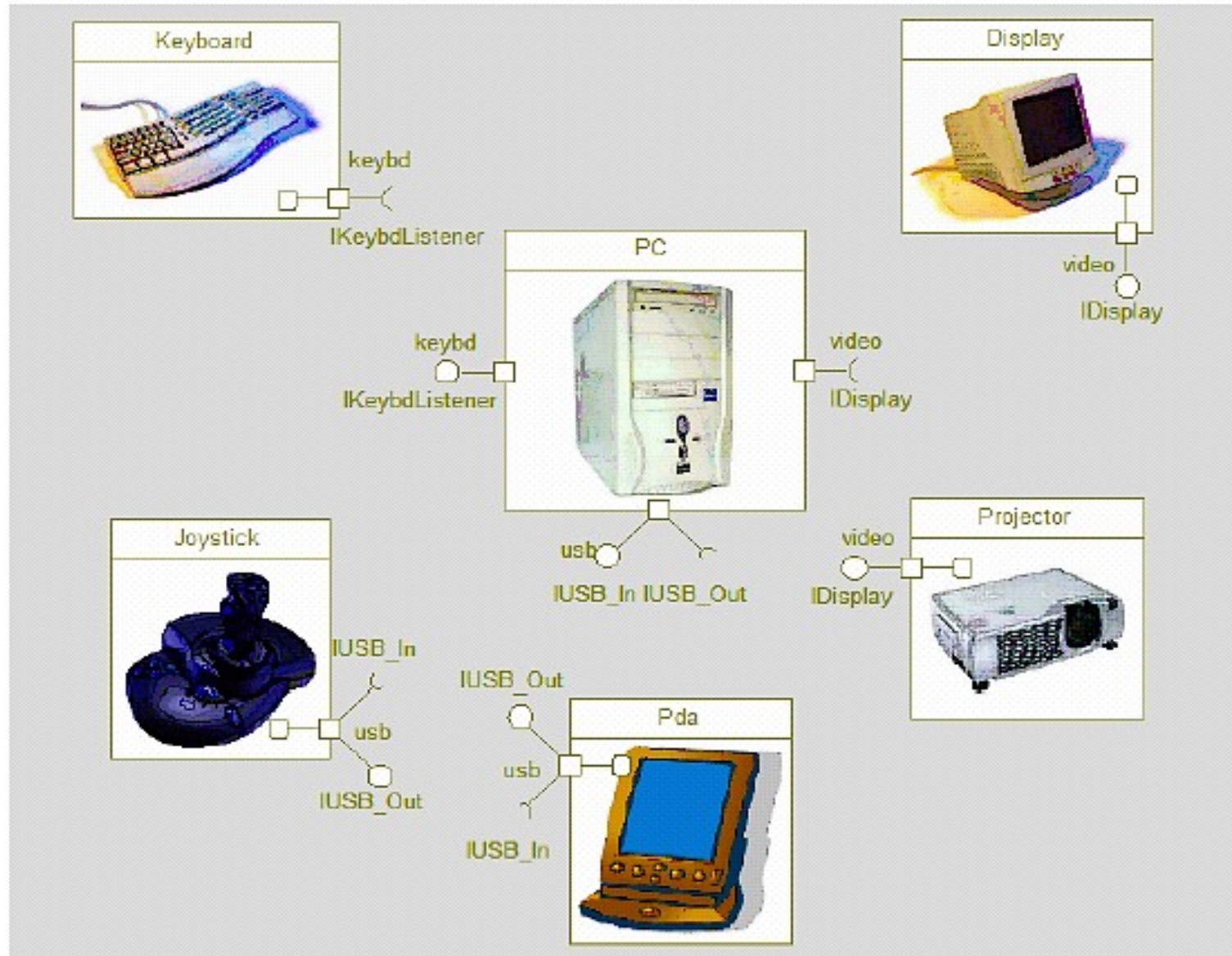
Components and their Relationships



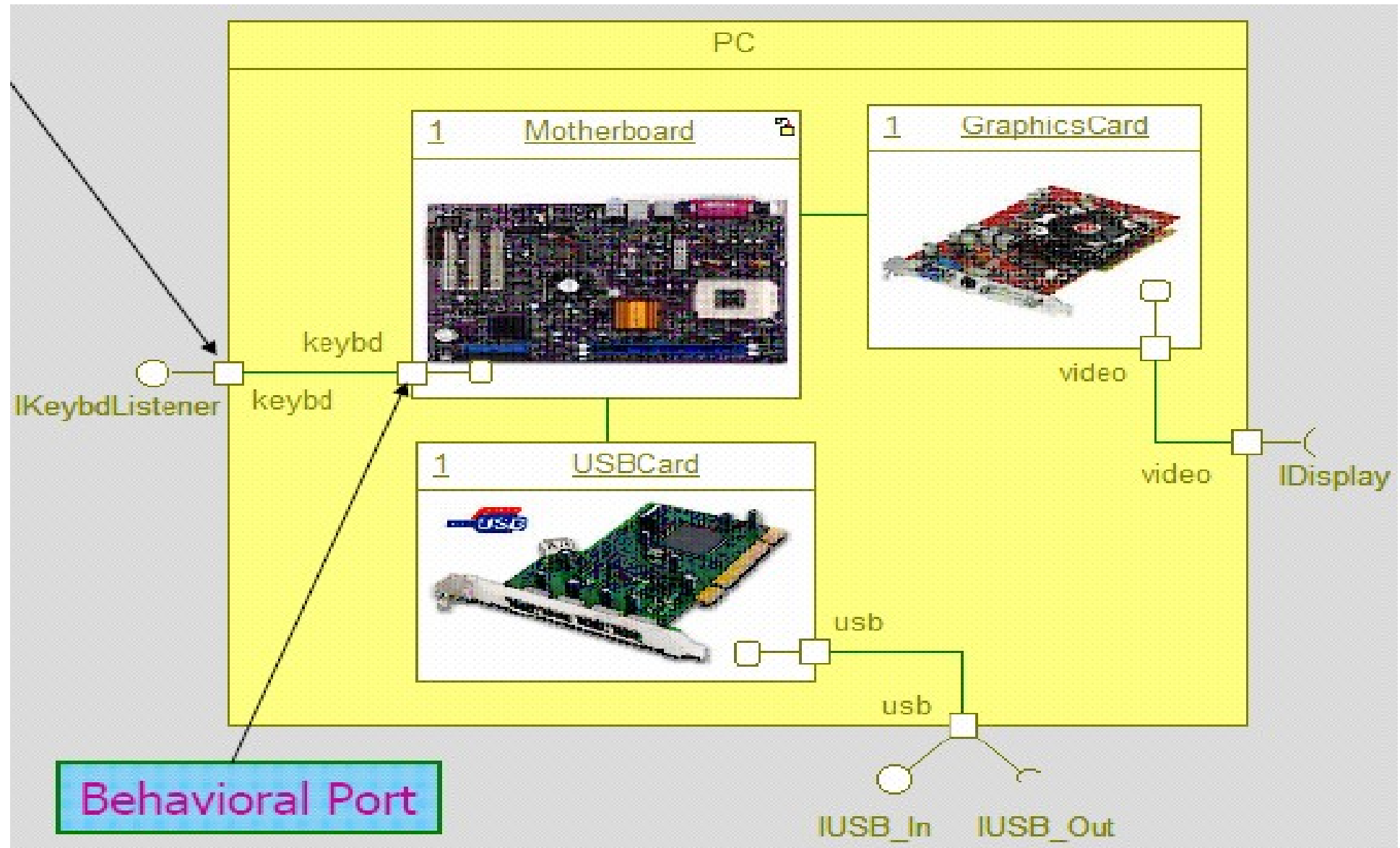
Component Diagram



COMPONENT DIAGRAM



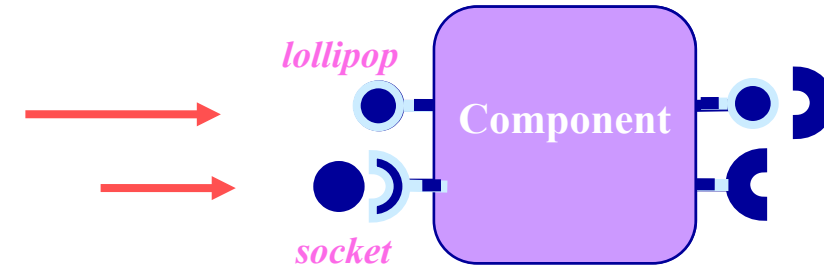
Component Diagram – another example



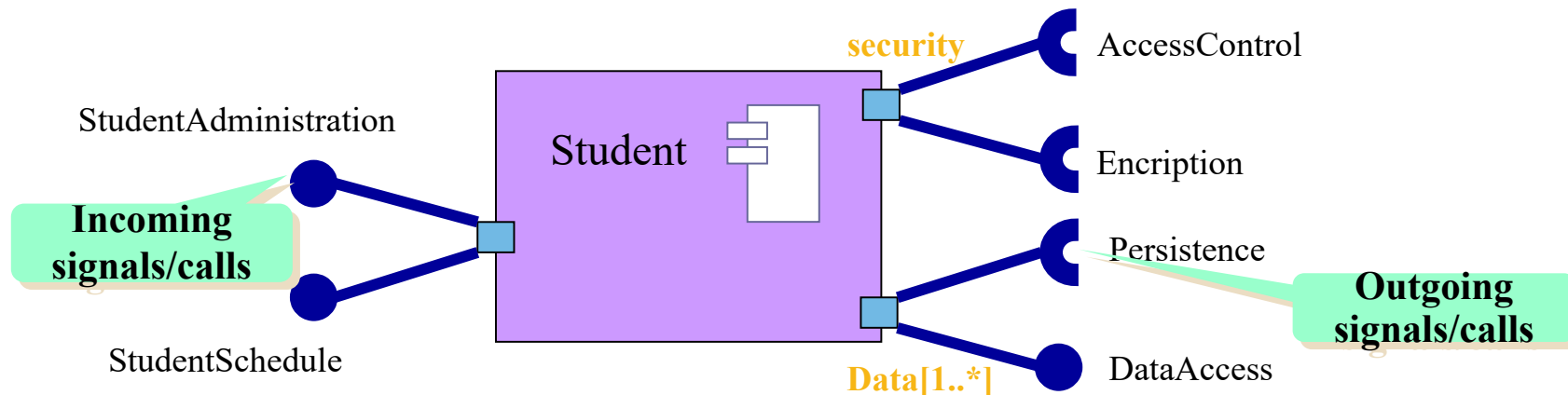
COMPONENT DIAGRAM

- Component is Logical unit block of the system, a slightly highest abstraction than class.
- Explicit description of *interfaces*:
 - ❑ provided services to other components
 - ❑ requested services from other components

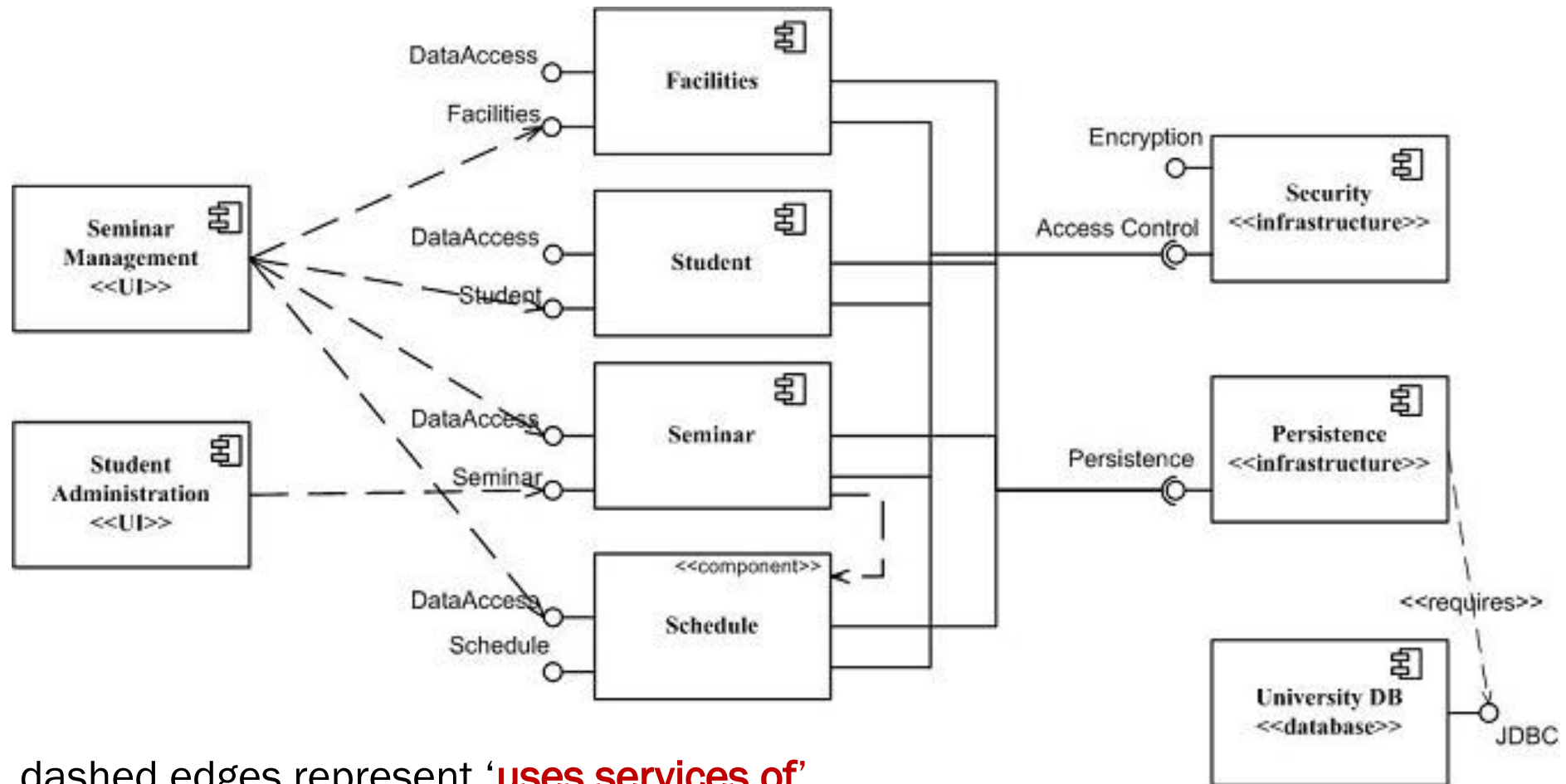
COMPONENT DIAGRAM



- An interface describes a group of operations used (required by) or created (provided) by a component
- Is a collection of 1..* methods, and 0..* attributes
- Full circle represents an interface created or provided by a component
- Semi circle represents a required interface
- Interfaces can consist of synchronous and / or asynchronous operations
- A **port (square)** is an interaction point between the component and its environment, it is often used to expose required and provided interfaces of a component
- Can be named; Can support uni-directional (either provide or require) or bi-directional (both provide and require) communication; Can support multiple interfaces.



COMPONENT DIAGRAM:



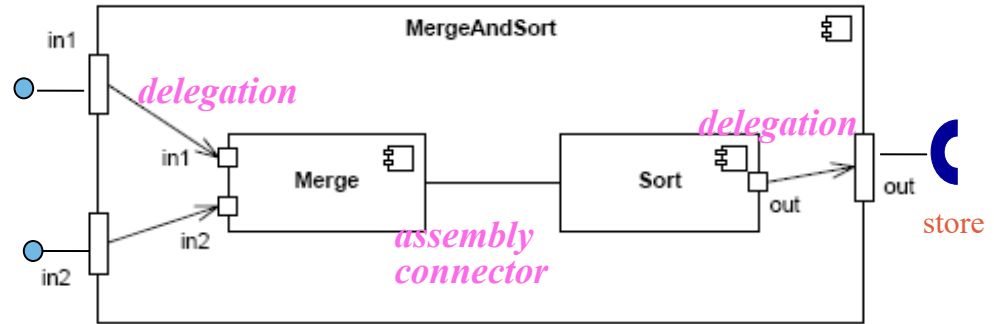
- since, dashed edges represent '**uses services of**'
 - thus, Seminar management component uses facility component ()

COMPONENT DIAGRAM – CONNECTOR & ANOTHER EXAMPLE

- ❑ a connector: just a link between two or more connectable elements (e.g., ports or interfaces)
- ❑ 2 kinds of connectors: *assembly* and *delegation*. For “wiring”
 - ❑ An *assembly* connector: a binding between a provided interface and a required interface (or ports) that indicates that one component provides the services required by another; *simple line/ball-and-socket/lollipop-socket notation*
 - ❑ A *delegation* connector binds a component’s external behavior (as specified at a port) to an internal realization of that behavior by one of its parts (*provide-provide, request-request*).



External View of a Component with Ports



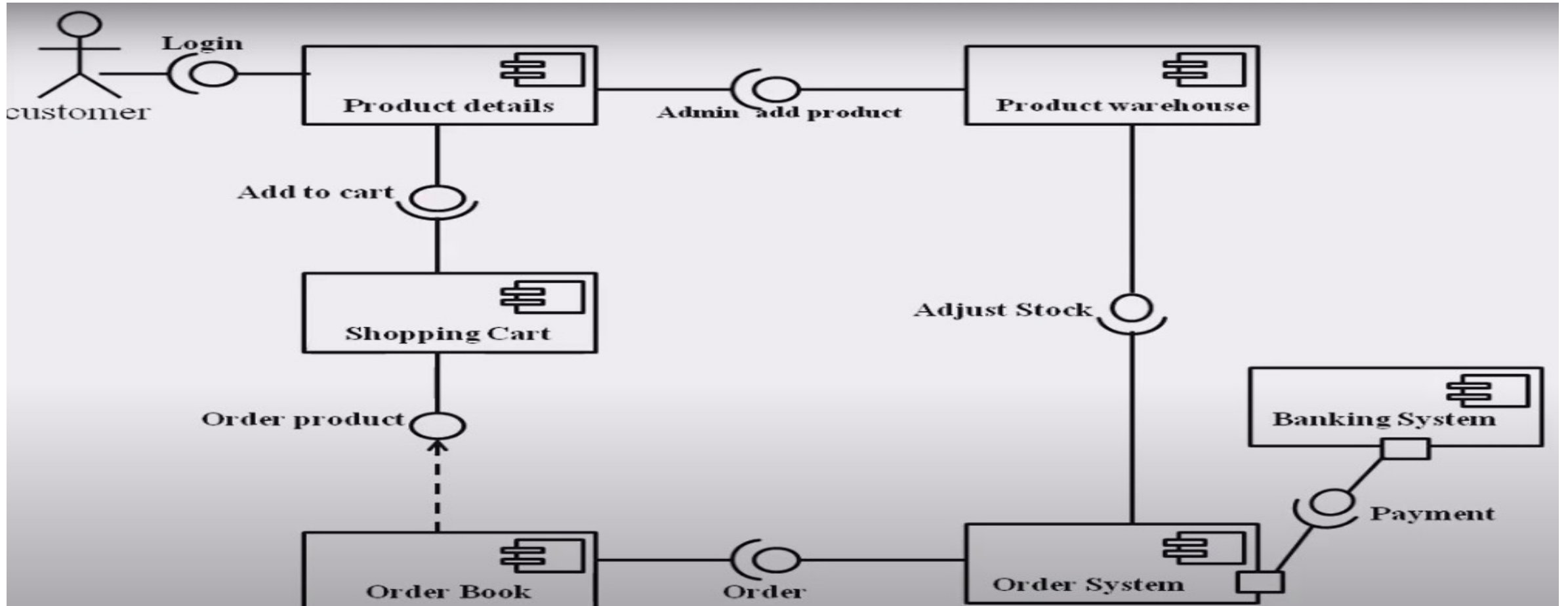
Internal View of a Component with Ports

Left delegation: direction of arrowhead indicates “provides”

Right delegation: direction of arrowhead indicates “requests”

So, what levels of abstractions for connections?

EASY UML COMPONENT DIAGRAM FOR ONLINE SHOPPING SYSTEM



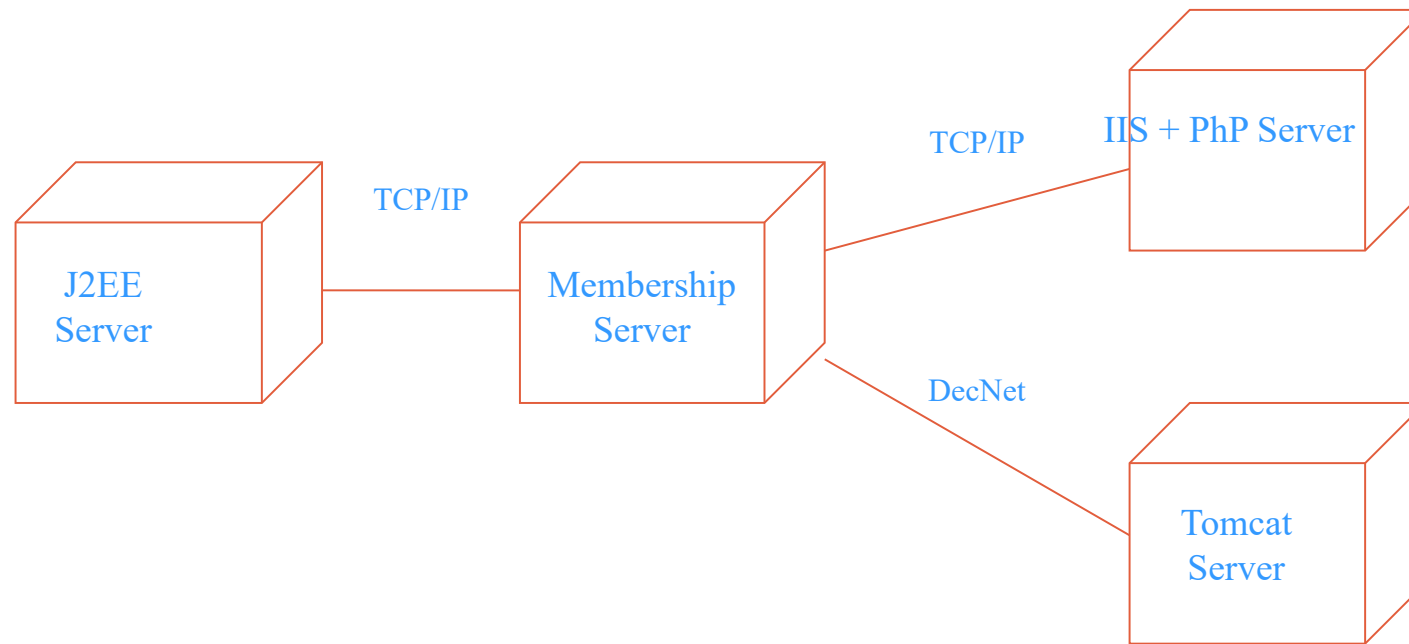
DEPLOYMENT DIAGRAMS



DEPLOYMENT DIAGRAM

- Provides Static deployment view of a system
- Unlike the other diagrams, it's the physical allocation of **components** to a **computational units**. **Think for example about The client-server architecture**
 - Define which component goes onto the client and which component go onto the server
- **Node** corresponds to **computational unit**(specific device)
- **Edge** corresponds to **communication**
- Shows a set of *processing* nodes and their relationships.
- Represents the static deployment view of an *architecture*.
- Nodes typically enclose one or more **components**.

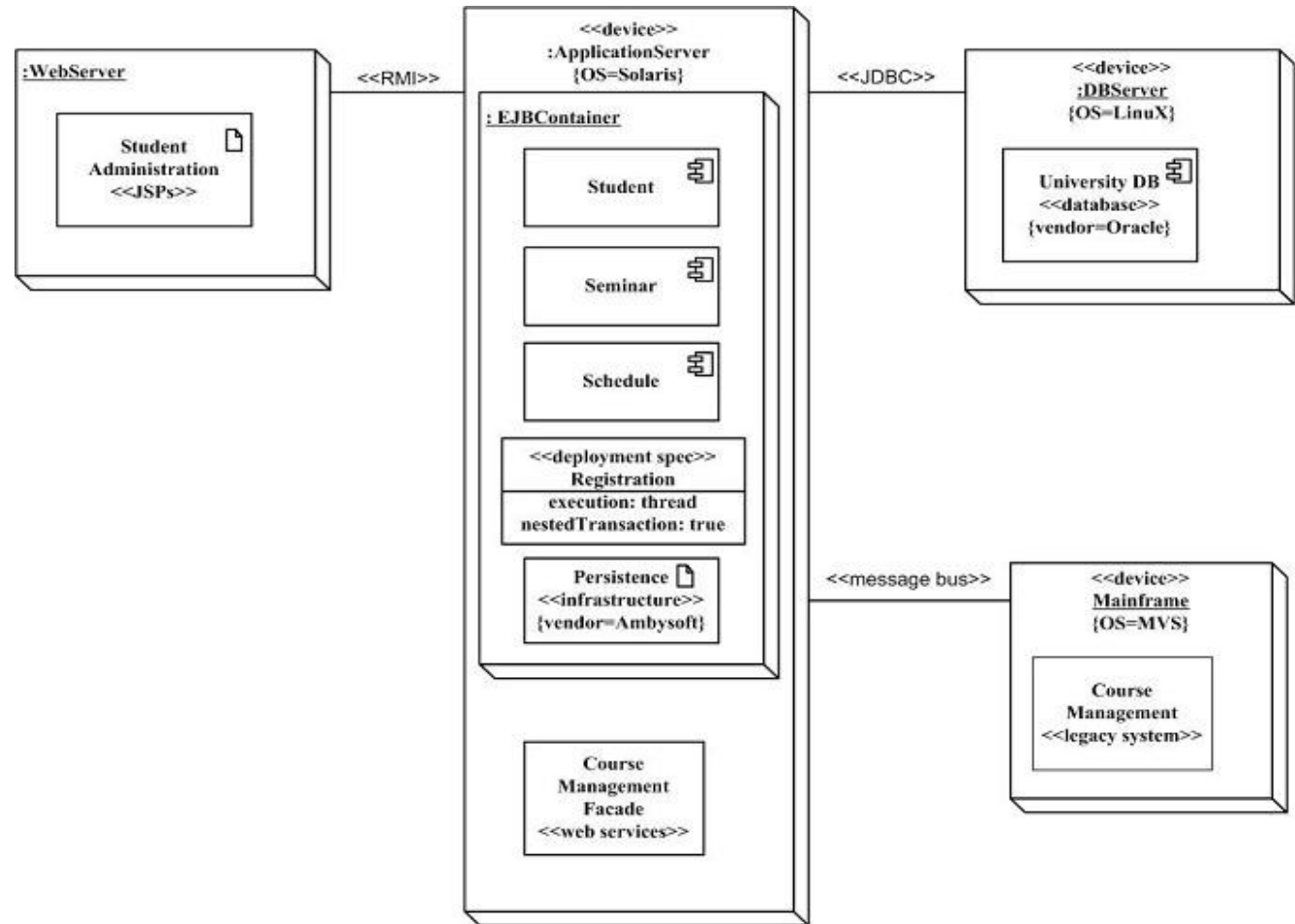
DEPLOYMENT DIAGRAM



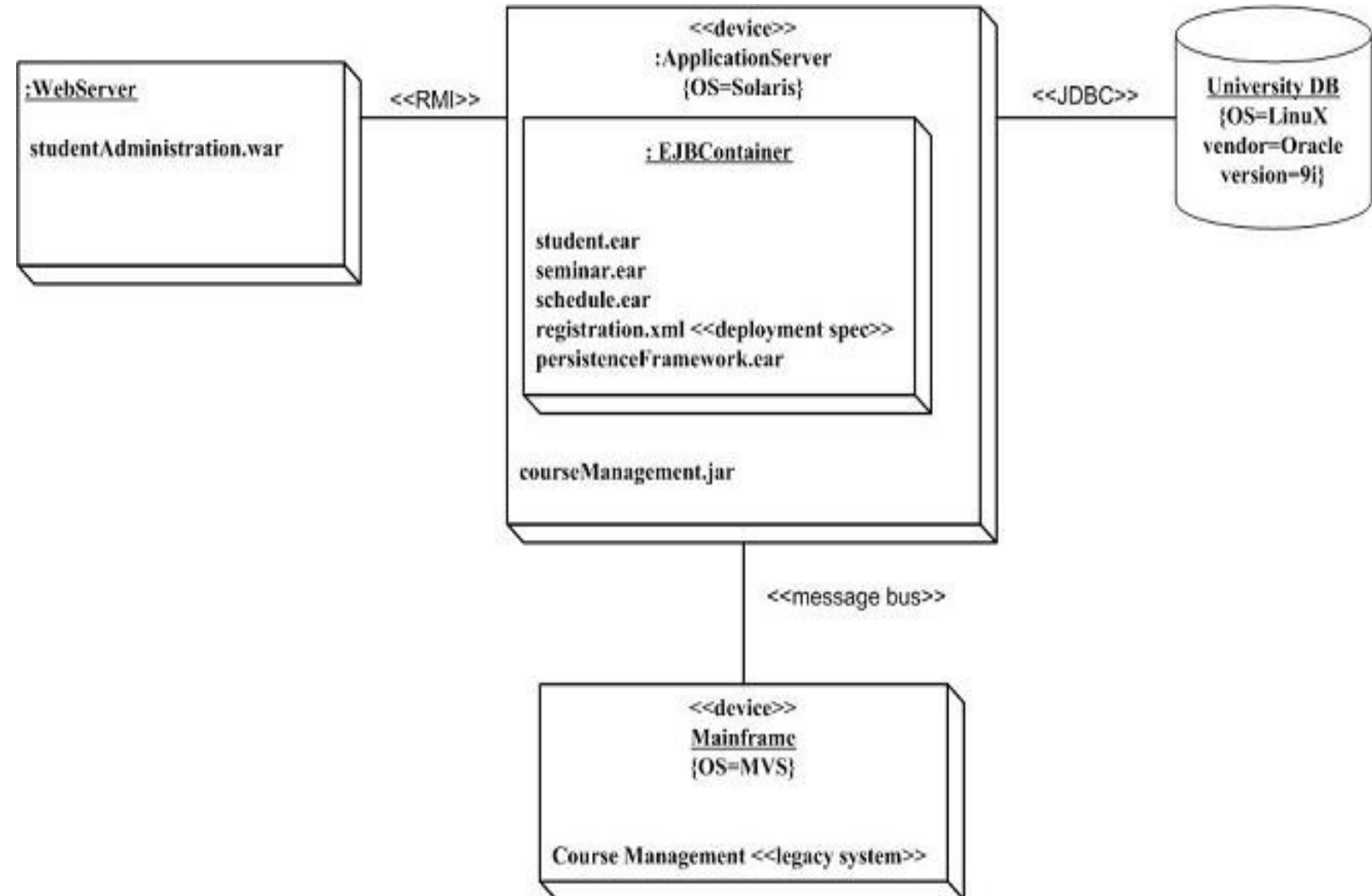
STRUCTURAL DIAGRAMS - DEPLOYMENT DIAGRAM

Student administration application

- ❑ Physical nodes - stereotype *device*
- ❑ *WebServer* - physical device or software artifact
- ❑ *RMI/message bus*: connection type
- ❑ Nodes can contain other nodes or software artifacts recursively
- ❑ Deployment specs: configuration files: name and properties



STRUCTURAL DIAGRAMS - DEPLOYMENT DIAGRAM



Is this better?

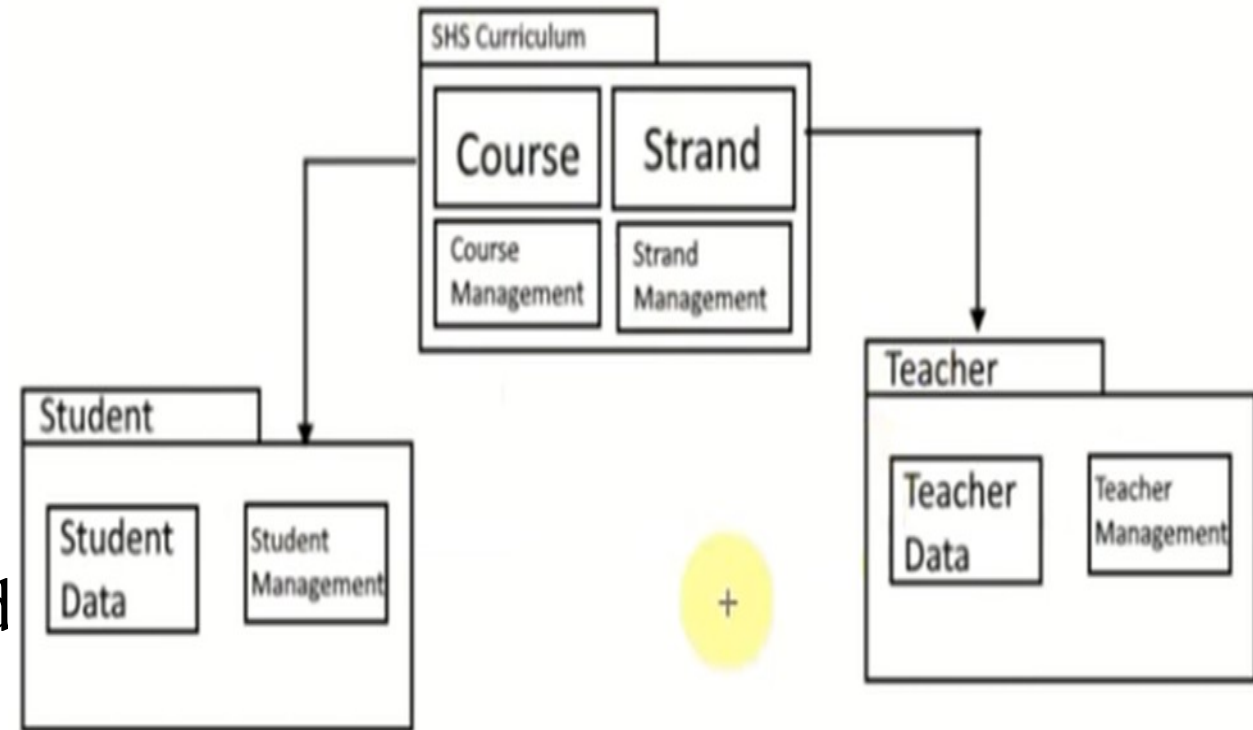
- ☐ More concrete
- ☐ Implementation-oriented

PACKAGE DIAGRAMS



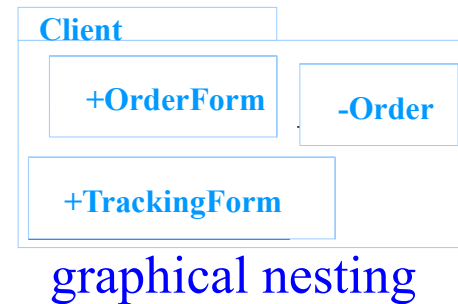
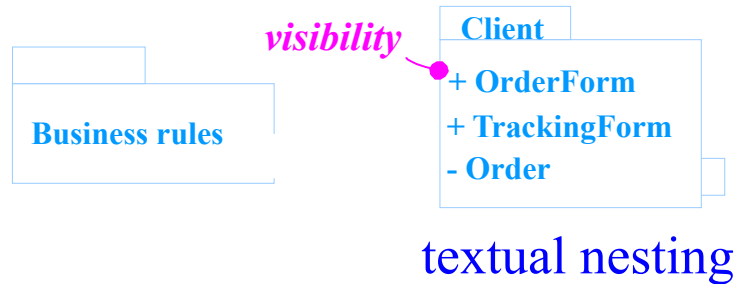
PACKAGES

- Package is a mechanism
 - for organizing similar elements
 - (classes or sub systems) into groups.
- Show Dependency among classes
- Here is a simplified version of package d

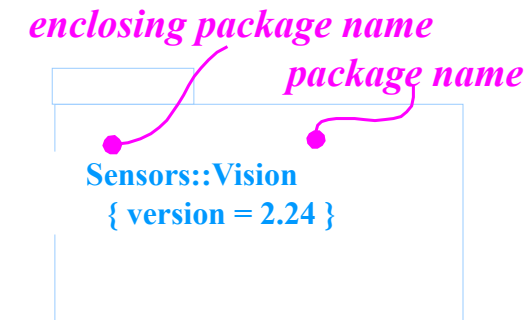


PACKAGES

simple names



path names

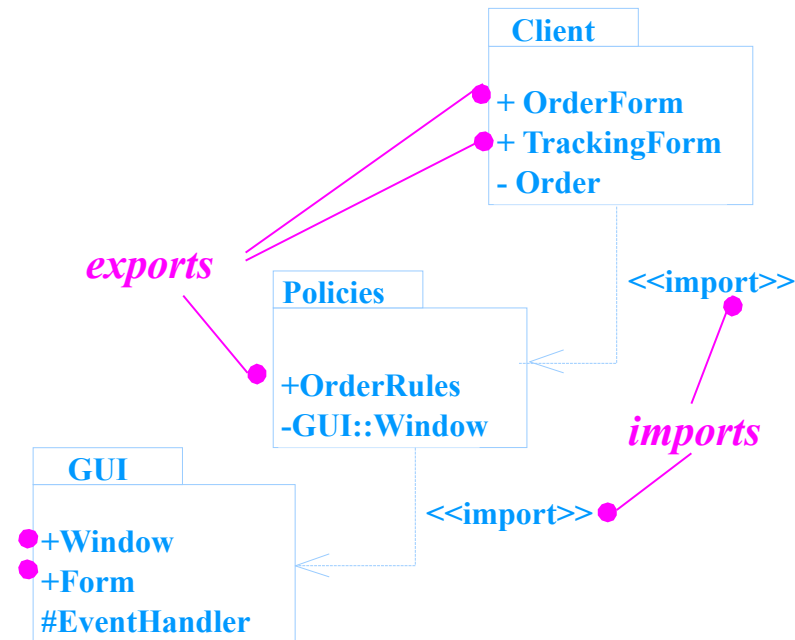
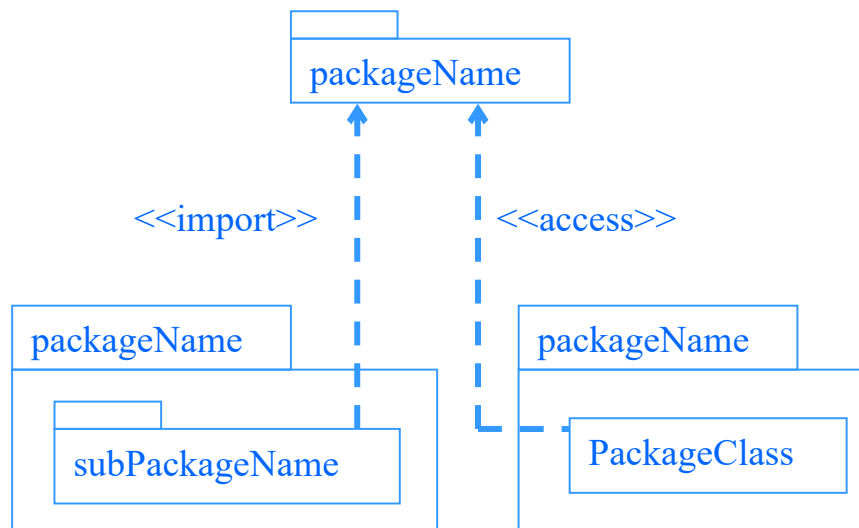


Visibility

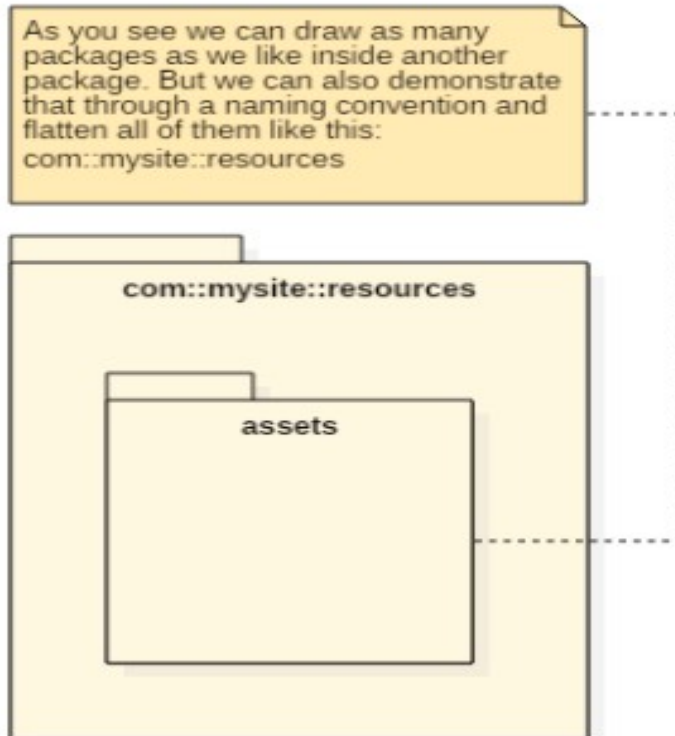
- Packages that are friends to another may see all the elements of that package, no matter what their visibility.
- If an element is visible within a package, it is visible within all packages nested inside the package.

DEPENDENCY –AMONG PACKAGES

- Two Stereotypes of Dependency Among Packages:
 - *access*: the source package is granted the right to reference the elements of the target package (*:: convention*)
 - *import*: a kind of access; the *public* contents of the target package enter the flat namespace of the source as if they had been declared in the source

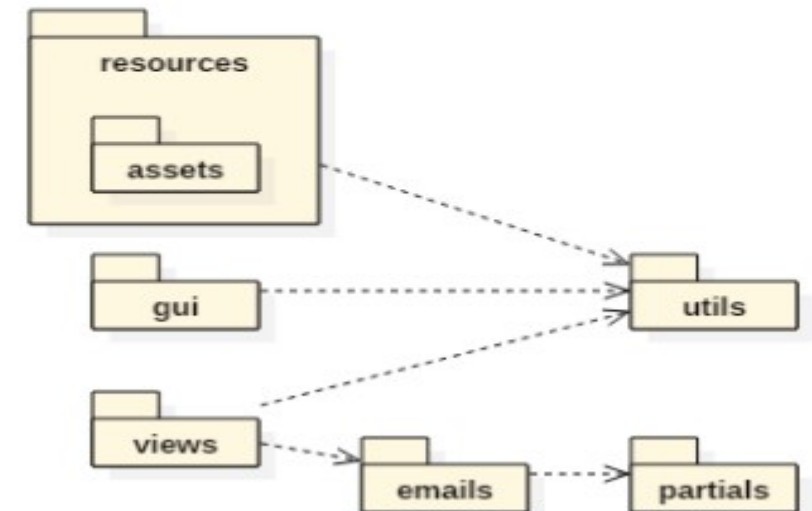


DEPENDENCY AMONG PACKAGES

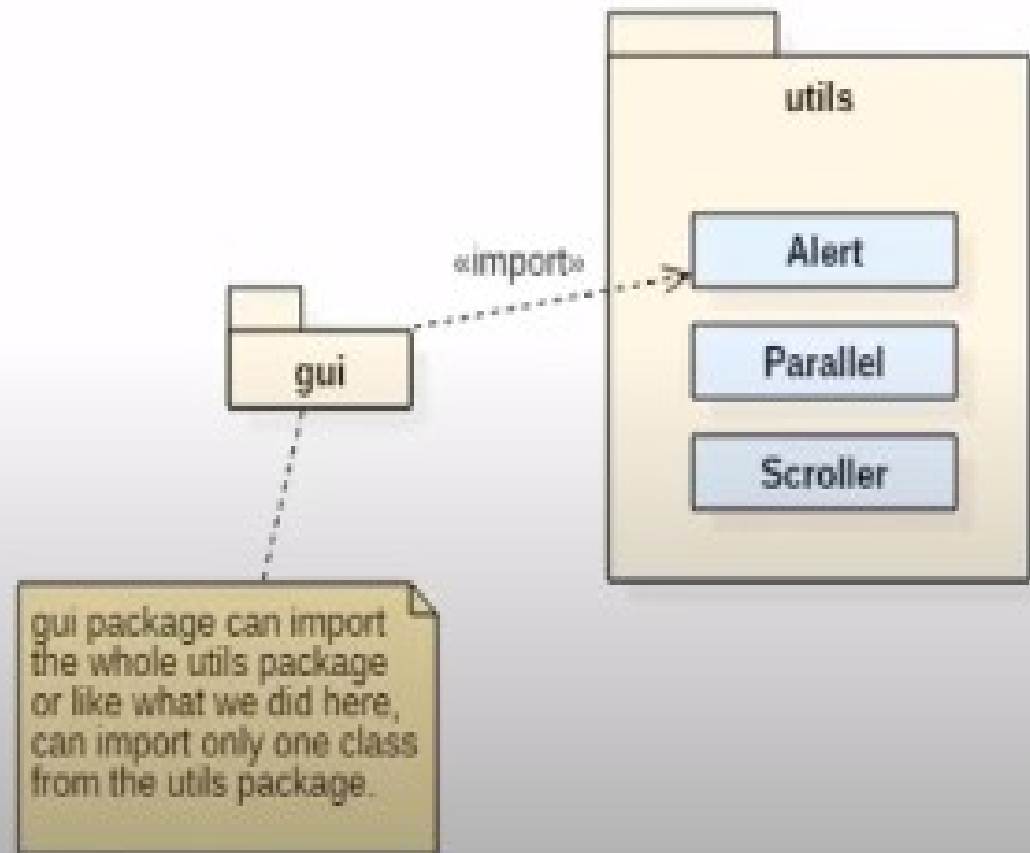
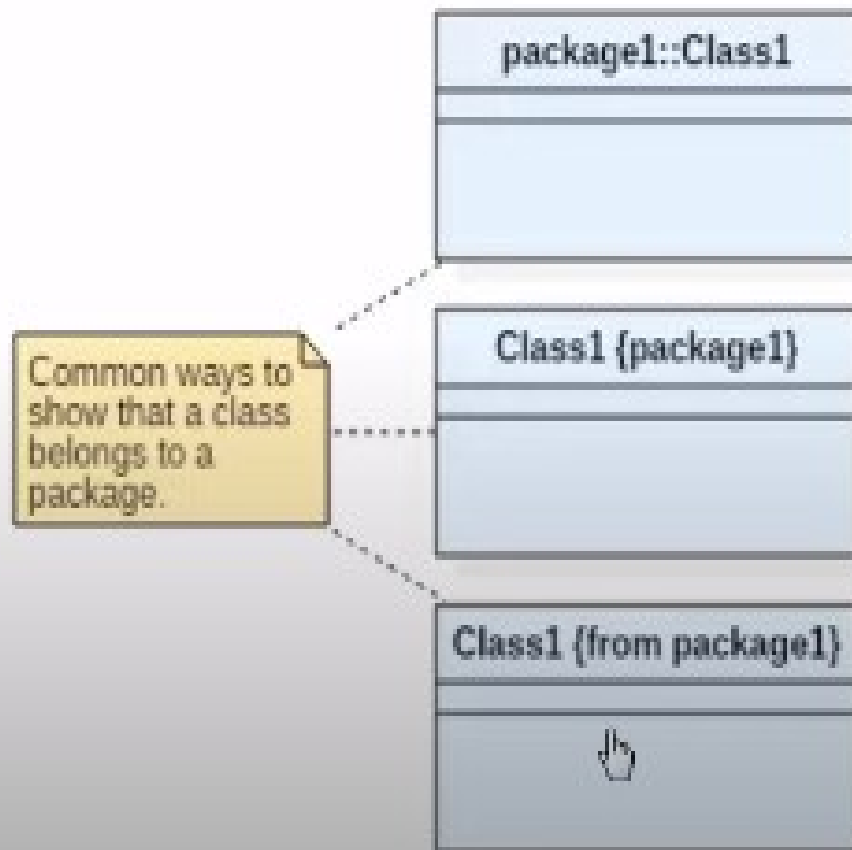


Package Diagram

And that is how we draw a Package Diagram and show each package's dependencies.



DEPENDENCY AMONG PACKAGES



ARCHITECTURAL PATTERNS (**READING ASSIGNMENT**)

- Software architecture is a pictorial representation of the IT system that communicates the following:
 - The organization of the system;
 - Different components with their functions;
 - How the components interact with each other.
- The schematic representation is an outcome of the design principles software architects use and the decisions they make.
- A typical software architecture requires decisions concerning security, performance, manageability, etc.
 - **The Multi-Layer architectural pattern**
 - **The Client–Server and other distributed architectural patterns**
 - **The Broker architectural pattern**
 - **The Pipe-and-Filter architectural pattern**
 - **The Model–View–Controller (MVC) architectural pattern**



WHY SHOULD YOU FOCUS ON SOFTWARE ARCHITECTURE?

- You need to get the software architecture right due to the following reasons:
 - A software development team makes the first set of decisions during the software architecture pattern selection.
 - Only the business requirements are available before this stage.
 - The first set of decisions are crucial in any project since sub-optimal decisions might result in budget and schedule overrun.
 - Your business stakeholders need to know about your progress.
 - Software architecture acts as a visual communication tool, therefore they can understand the system you are building.
 - Software architecture models make it easy to reuse them in other projects since you now know the decisions you made and the various trade-offs.

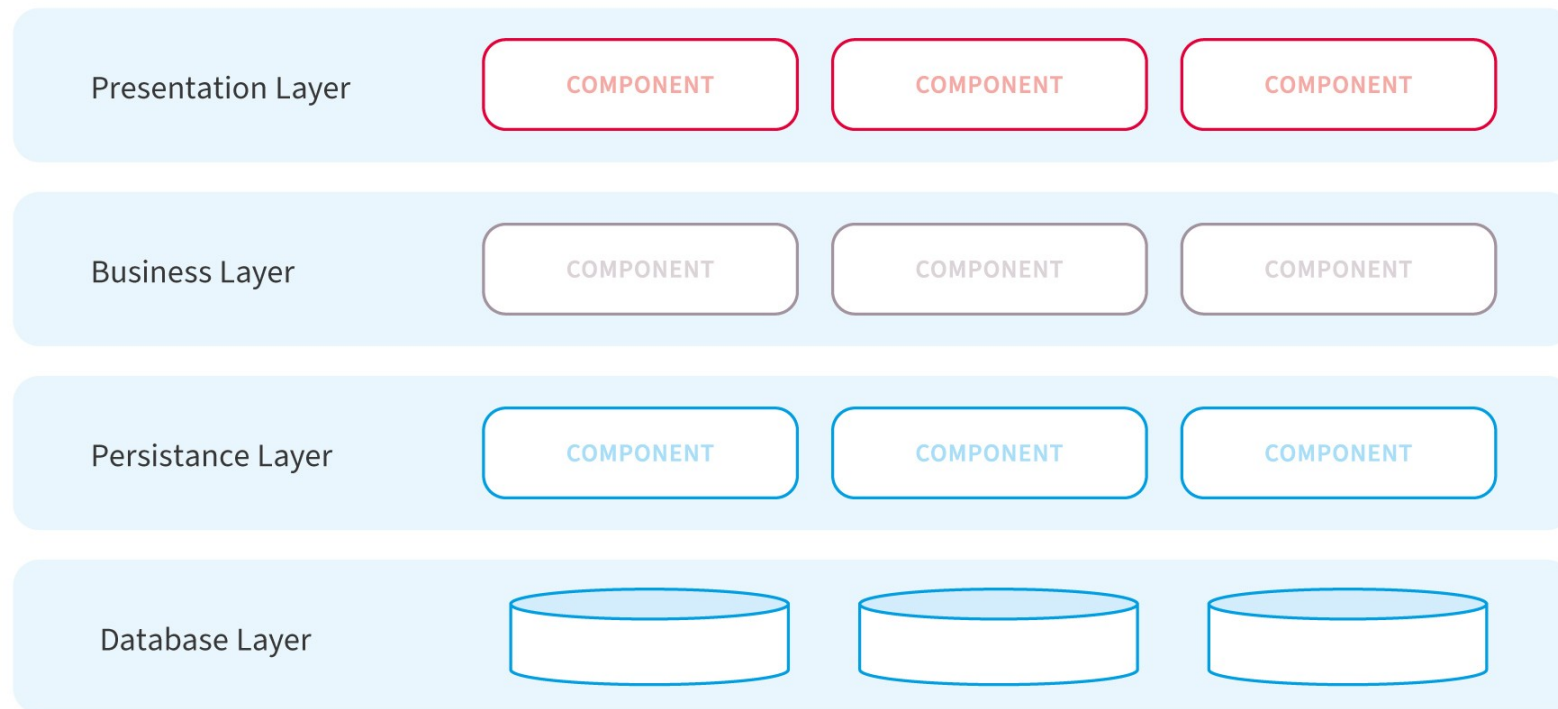
THE MULTI-LAYER ARCHITECTURAL PATTERN

- In layered systems each layer communicates only with the layers below it
 - Each layer has a well-defined **API**, defining the services it provides
- A complex system can be built by superimposing layers at increasing levels of abstraction.
- The Multi-Layer architectural pattern makes it possible to replace a layer by an improved version
- It's one of the most **commonly used** patterns where the **code is arranged** in **layers**.

THE MULTI-LAYER ARCHITECTURAL PATTERN

- **Key characteristics** of this pattern are as follows:
 - The outermost layer is where the data enters the system.
 - The data passes through the subsequent layers to reach the innermost layer, which is the database layer.
 - Simple implementations of this pattern have at least 3 layers, namely, a presentation layer, an application layer, and a data layer.
 - Users access the presentation layer using a GUI, whereas the application layer runs the business logic.
 - The data layer has a database for the storage and retrieval of data.
- This pattern has the following **advantages**:
 - **Maintaining** the software is **easy** since the tiers are segregated.
 - Development teams find it easy to manage the software infrastructure, therefore, it's **easy** to **develop large-scale web and cloud-hosted apps**.

THE MULTI-LAYER ARCHITECTURAL PATTERN



Layered pattern

THE CLIENT–SERVER AND OTHER DISTRIBUTED ARCHITECTURAL PATTERNS

- Is another commonly used one, where there are 2 entities.
 - It has a set of clients and a server.
- The following are key characteristics of this pattern:
 - Client components send requests to the server, which processes them and responds back.
 - When a server accepts a request from a client, it opens a connection with the client over a specific protocol.
 - Servers can be stateful or stateless.
 - A stateful server can receive multiple requests from clients.
 - It maintains a record of requests from the client, and this record is called a ‘session’.

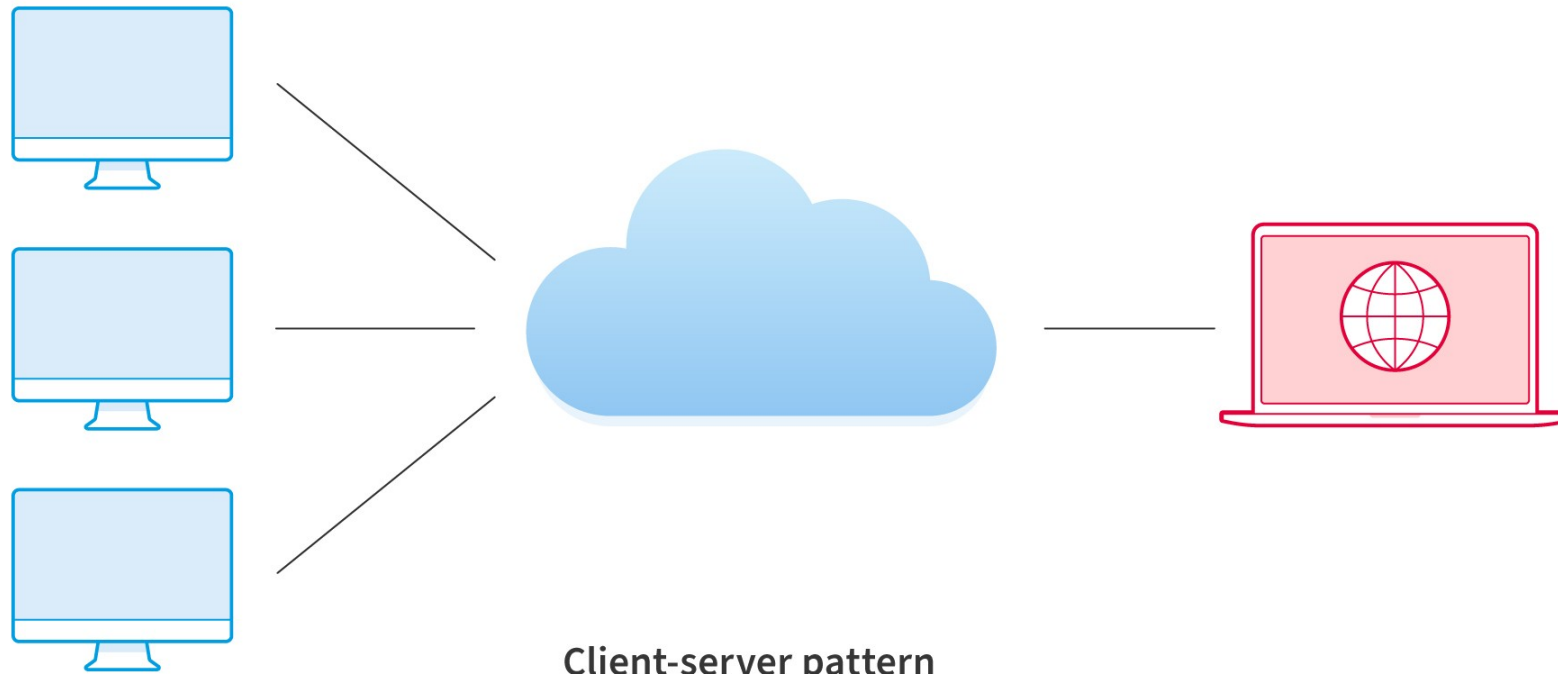


■ **Email** applications are good examples of this pattern.

■ The pattern has several **advantages**, as follows:

- Clients access data from a server using authorized access, which improves the **sharing of data**.
- Accessing a service is via a ‘user interface’ (**UI**), therefore, there’s no need to run terminal sessions or command prompts.
- Client-server applications can be built irrespective of the platform or technology stack.
- This is a **distributed model** with specific responsibilities for each component, which makes **maintenance easier**.

THE CLIENT-SER

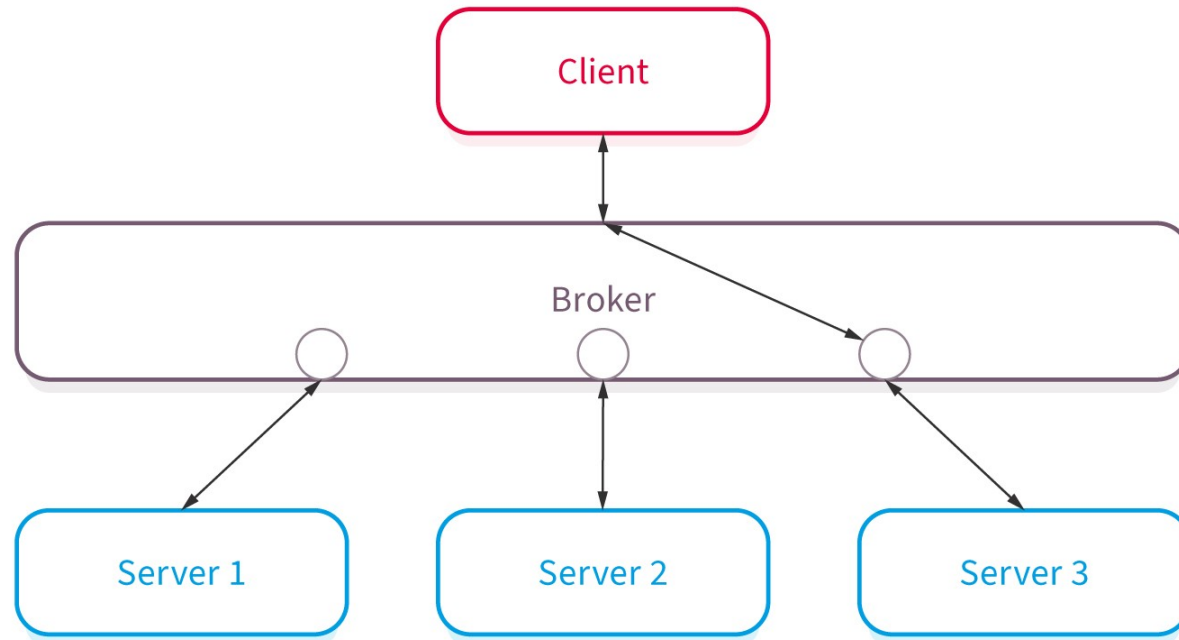


Client-server pattern

THE BROKER ARCHITECTURAL PATTERN

- Consider distributed systems with components that provide **different services independent** of each other.
- Independent components could be **heterogeneous** systems **on different servers**, however, clients still need their requests serviced.
 - “Broker architecture pattern” is a solution to this.
- It has the following broad **characteristics**:
 - A broker component **coordinates requests** and responses between **clients** and **servers**.
 - The broker has the **details of the servers** and the individual **services** they provide.
 - The main **components** of the broker architectural pattern are **clients**, **servers**, and **brokers**.
 - It also has bridges and proxies for clients and servers.
 - **Clients** send **requests**, and the **broker finds** the right **server** to route the request to. It also **sends** the **responses** back to the **clients**.

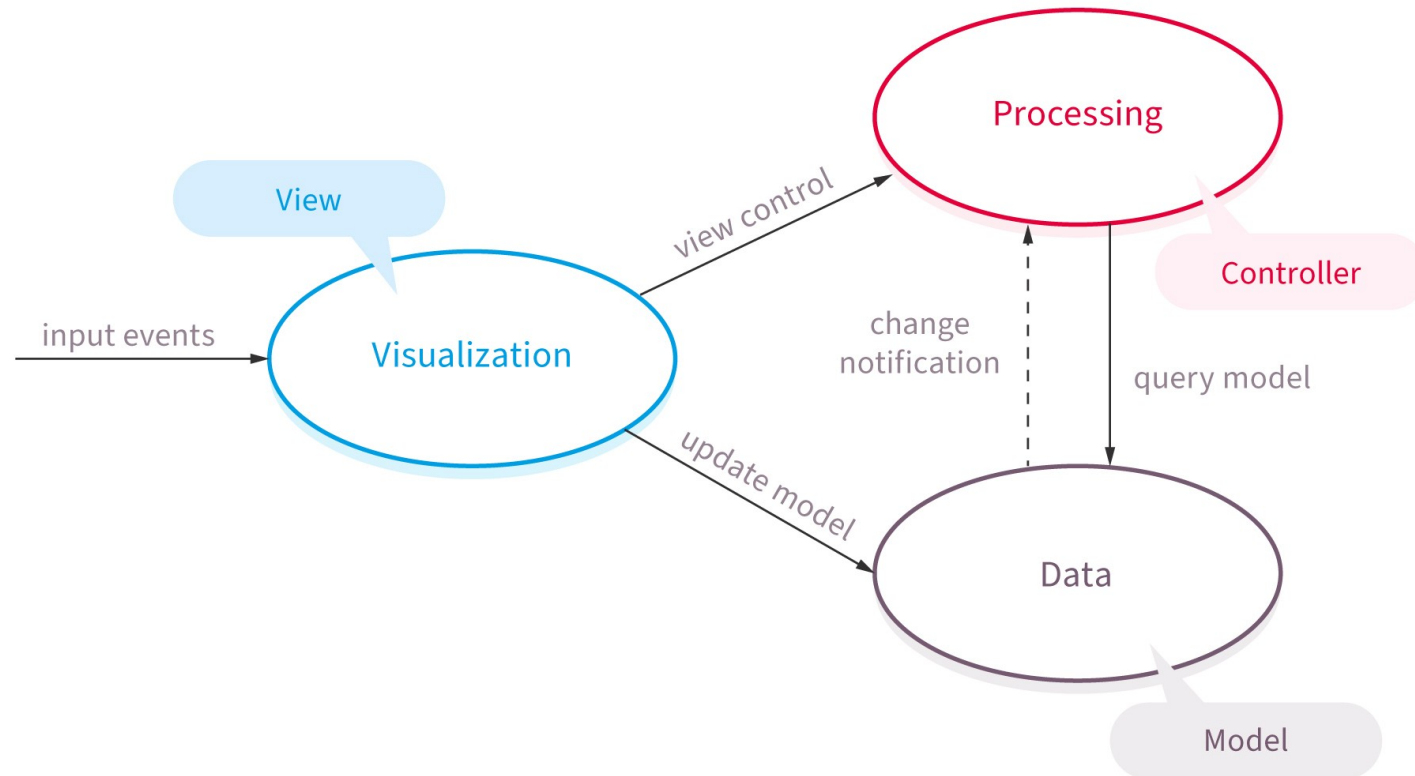
THE BROKER ARCHITECTURAL PATTERN



Broker pattern

- This pattern is popular and involves separating an applications' **data model**, **presentation layer**, and **control** aspects.
- Following are its' **characteristics**:
 - There are three **building blocks** here, namely, **model**, **view**, and **controller**.
 - The application **data** resides in the **model**.
 - **Users see** the application data through the **view**, however, the view can't influence what the user will do with the data.
 - The controller is the building block between the model and the view.
 - **View triggers** events, subsequently, the **controller acts** on it.
 - The action is typically a method **call** to the **model**.
 - The **response** is shown in the **view**.
- Its' **advantages**:
 - Using this model **expedites** the **development**.
 - Development teams can present multiple views to users.
 - Changes to the UI is common in web applications, however, the MVC pattern doesn't need changes for it.
 - The model doesn't format data before presenting to users, therefore, you can use this pattern with any interface.

THE MODEL–VIEW–CONTROLLER (MVC) ARCHITECTURAL PATTERN

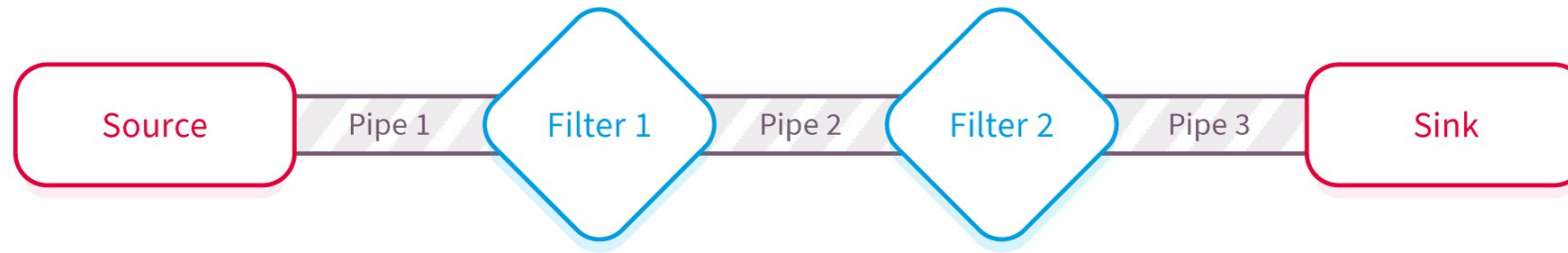


Model-view-controller pattern

THE PIPE-AND-FILTER ARCHITECTURAL PATTERN

- Suppose you have **complex processing** in hand.
 - You will likely **break** it down into **separate tasks** and **process** them **separately**.
 - This is where the “Pipe-filter” architecture pattern comes in use.
- Following characteristics distinguish it:
 - The code for each task is relatively small. You treat it as one independent ‘filter’.
 - You can **deploy**, **maintain**, **scale**, and **reuse** code **in each filter**.
 - The stream of **data** that each filter processes **pass** through ‘**pipes**’.
- **Compilers** often use this pattern, due to the following advantages:
 - There are repetitive steps such as reading the source code, parsing, generating code, etc. These can be easily organized as separate filters.
 - Each filter can perform its’ processing in parallel if the data input is arranged as streams using pipes.
 - It’s a resilient model since the pipeline can reschedule the work and assign to another instance of that filter.

THE PIPE-AND-FILTER ARCHITECTURAL PATTERN



Pipe-filter pattern



END